

BAB II

PRODUK

Bab ini merinci proses perancangan dan pengembangan produk aplikasi *mobile* "RumahTBC", yang bertujuan sebagai solusi digital terintegrasi untuk deteksi dini, kepatuhan, dan pemantauan pengobatan tuberkulosis. Pengembangan produk ini mencakup beberapa tahapan utama, mulai dari perancangan sistem, pembuatan aplikasi, hingga pengujian fungsionalitas.

2.1 Perancangan

Tahap perancangan merupakan fondasi dalam pengembangan sistem untuk memastikan aplikasi yang dibangun sesuai dengan kebutuhan pengguna dan tujuan penelitian. Proses ini mengubah analisis kebutuhan dari data penelitian menjadi sebuah model kerja yang detail sebelum masuk ke tahap implementasi.

2.1.1 Perancangan Aplikasi

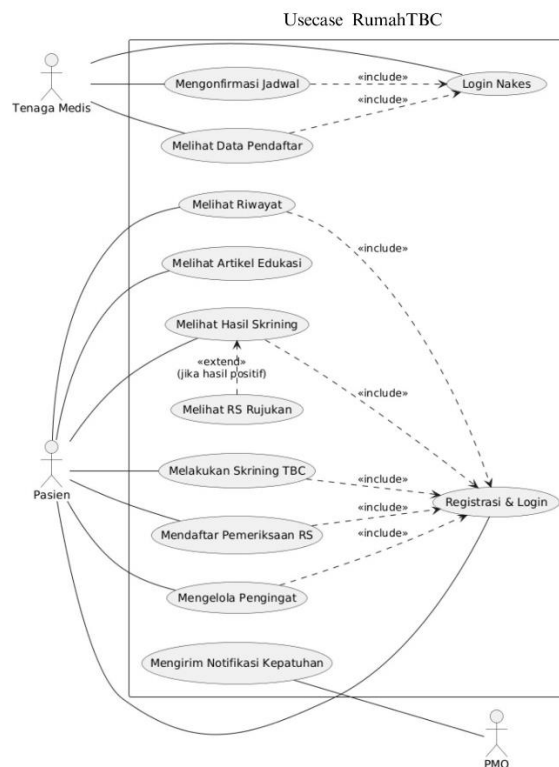
Perancangan aplikasi "RumahTBC" dilakukan secara terstruktur menggunakan berbagai alat pemodelan untuk memvisualisasikan alur kerja, interaksi pengguna, dan arsitektur sistem. Pemodelan ini penting untuk memahami sistem yang kompleks secara menyeluruh sebelum proses *coding* dimulai[29].

1. Use Case Diagram

Use Case Diagram digunakan untuk memodelkan fungsionalitas sistem dari sudut pandang pengguna (*actor*). Diagram ini mendeskripsikan interaksi antara pengguna (pasien, tenaga medis dan Pendamping Minum Obat/PMO) dengan fitur-fitur utama yang tersedia di aplikasi "RumahTBC". Diagram ini membantu

mengidentifikasi fungsi apa saja yang ada di dalam sistem dan siapa yang berhak menggunakannya [30].

Untuk aplikasi "RumahTBC", seperti yang bisa dilihat pada Gambar 2.1 *actor* utamanya adalah Pasien, Tenaga Medis dan PMO.



Gambar 2.1 *Use Case Diagram* Aplikasi

Deskripsi Aktor dan *Use Case*:

a. Aktor: Pasien

- 1) *Registrasi & Login*: Melakukan pendaftaran akun baru dan masuk ke aplikasi.
- 2) *Melakukan Skrining TBC*: Mengisi kuesioner skrining untuk deteksi dini.

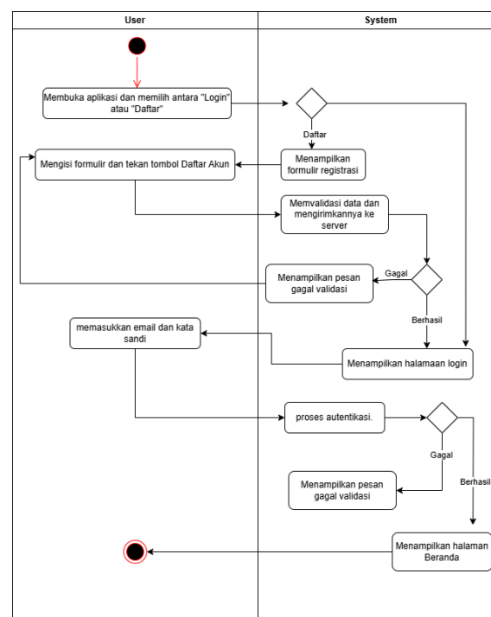
- 3) Melihat Hasil Skrining: Melihat hasil skrining yang bersifat biner (Positif/Negatif Skrining). *Use case* ini memiliki relasi <<*extend*>> ke Melihat RS Rujukan jika hasilnya positif.
 - 4) Mendaftar Pemeriksaan RS: Melakukan pendaftaran untuk pemeriksaan di RS rujukan.
 - 5) Mengelola Pengingat: Mengatur jadwal minum obat dan jadwal kontrol.
 - 6) Melihat Riwayat: Mengakses riwayat skrining mandiri dan riwayat pemeriksaan medis dari faskes.
 - 7) Melihat Artikel Edukasi: Membaca artikel terkait TBC.
- b. Aktor: Tenaga Medis
- 1) *Login*: Masuk ke sistem (diasumsikan melalui platform terpisah seperti *web dashboard*).
 - 2) Melihat Data Pendaftar: Melihat daftar pasien yang mendaftar untuk pemeriksaan.
 - 3) Mengakses Hasil Skrining Pasien: Melihat hasil skrining pasien sebagai data rujukan awal.
 - 4) Mengonfirmasi Jadwal: Menyetujui atau mengubah jadwal pemeriksaan yang diajukan pasien.
- c. Aktor: PMO (Pendamping Minum Obat)
- 1) Menerima Notifikasi Kepatuhan: Menerima notifikasi melalui WhatsApp terkait status minum obat pasien (misalnya, jika pasien terlewat jadwal). PMO adalah aktor pasif yang menerima informasi dari sistem.

2. Activity Diagram

Activity Diagram menggambarkan alur kerja (*workflow*) dari setiap proses bisnis yang ada di dalam aplikasi. Berbeda dari *Flowchart*, diagram ini menggambarkan berbagai aliran aktivitas dalam sistem, bagaimana setiap alur dimulai, keputusan yang mungkin terjadi, dan bagaimana alur tersebut berakhir. Berikut adalah rincian untuk setiap aktivitas:

a. Activity Diagram Register dan Login

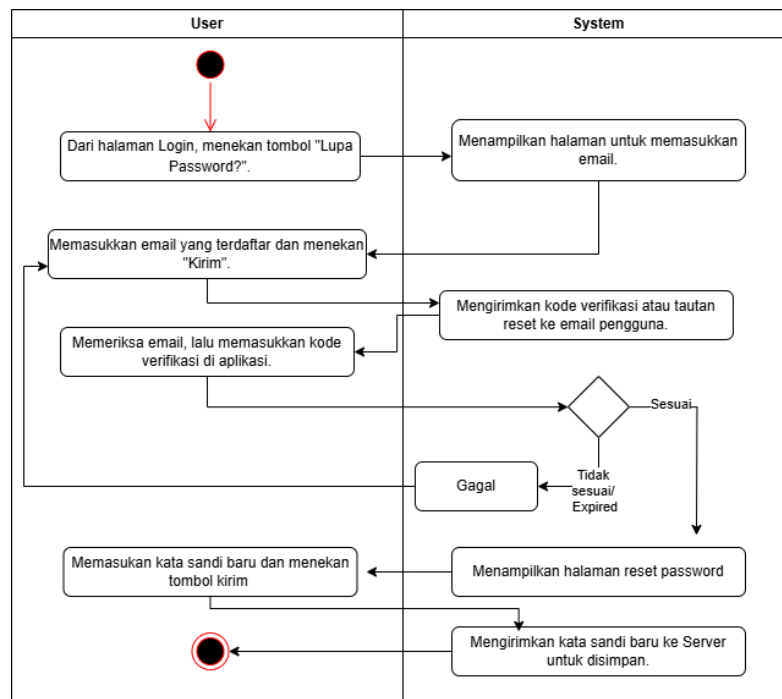
Diagram ini menggambarkan alur kerja saat pengguna mendaftarkan akun baru dan masuk ke dalam aplikasi. Proses dimulai saat pengguna memilih opsi "Daftar" atau "Login", kemudian mengisi data pada formulir. Sistem akan melakukan validasi terhadap input pengguna. Jika data valid, pengguna akan diarahkan ke halaman utama aplikasi; jika tidak, sistem akan menampilkan pesan kesalahan untuk perbaikan. Alur antarmuka yang dimodelkan oleh *activity diagram* ini dapat dilihat pada Gambar 2.2.



Gambar 2.2 Activity Diagram Register dan Login

b. *Activity Diagram Lupa Password*

Diagram ini menjelaskan langkah-langkah yang harus diikuti pengguna jika lupa kata sandi. Alur dimulai saat pengguna menekan tombol "Lupa Password", kemudian memasukkan email terdaftar. Sistem akan mengirimkan kode verifikasi atau tautan reset ke email tersebut. Pengguna kemudian membuka email, memasukkan kode, lalu membuat kata sandi baru untuk memulihkan akses akun. Ilustrasi alur antarmuka untuk fitur lupa password dapat dilihat pada Gambar 2.3.



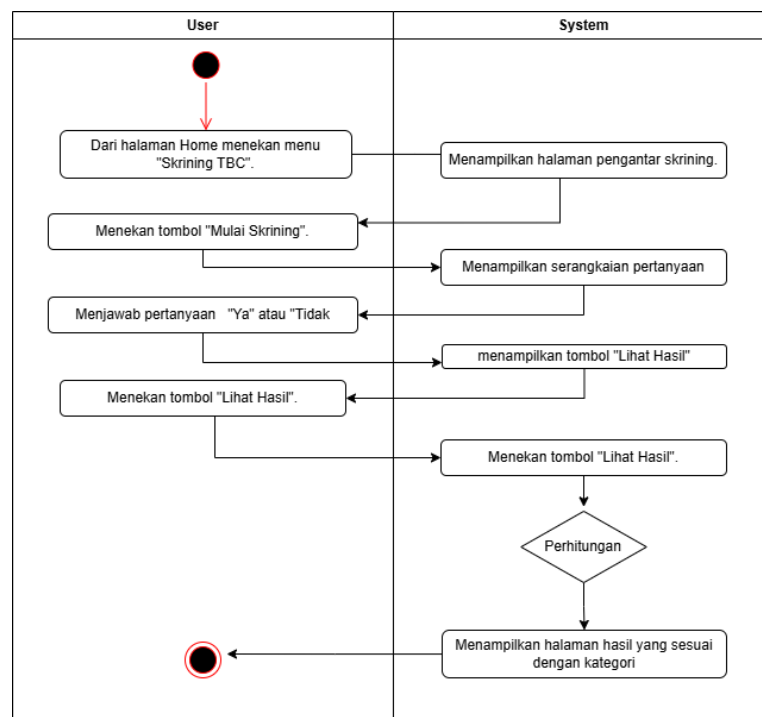
Gambar 2.3 *Activity Diagram Lupa Password*

c. *Activity Diagram Deteksi Dini/ Skrining TBC*

Diagram ini memperlihatkan alur pengguna saat menjawab serangkaian pertanyaan skrining, proses sistem dalam menentukan hasil, hingga menampilkan hasil deteksi. Alur dimulai dari halaman Home menuju fitur

skrining, di mana pengguna menjawab 16 pertanyaan terkait tanda, gejala, dan faktor risiko TBC.

Setelah itu, sistem akan memproses jawaban menggunakan aturan keputusan ganda (*dual decision rule*). Pengguna akan diklasifikasikan sebagai "**Positif Skrining**" jika memenuhi salah satu dari tiga kriteria: (1) mengalami batuk selama 2 minggu atau lebih, (2) memiliki riwayat kontak dengan penderita TBC, atau (3) memiliki total skor gabungan ≥ 7 . Jika tidak ada kriteria yang terpenuhi, hasilnya adalah "**Negatif Skrining**". Ilustrasi alur untuk fitur skrining dapat dilihat pada Gambar 2.4.

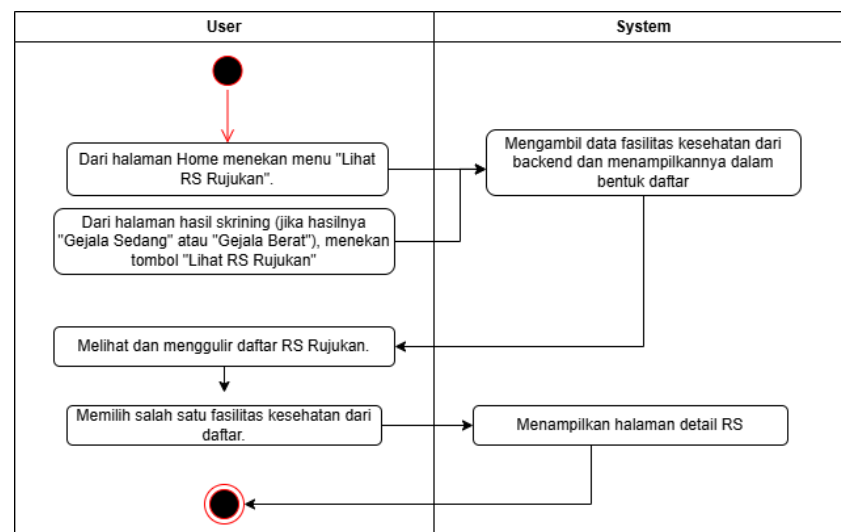


Gambar 2.4 Activity Diagram Deteksi Dini/ Skrining TBC

d. Activity Diagram RS Rujukan

Diagram ini menggambarkan proses saat pengguna melihat daftar rumah sakit rujukan terdekat setelah mendapatkan hasil skrining "**Positif**

Skrining". Alur dimulai dari halaman hasil deteksi, di mana pengguna menekan tombol untuk melihat RS Rujukan. Sistem kemudian menampilkan daftar fasilitas kesehatan, yang dapat dipilih pengguna untuk melihat detail alamat, kontak, dan peta lokasi. Alur antarmuka untuk fitur RS Rujukan dapat dilihat pada Gambar 2.5.



Gambar 2.5 Activity Diagram RS Rujukan

e. Activity Diagram Riwayat Pemeriksaan

Diagram ini menjelaskan bagaimana pengguna mengakses dan melihat seluruh riwayat hasil pemeriksaan resmi yang datanya dikirimkan oleh rumah sakit mitra. Alur dimulai dari pengguna memilih menu "Riwayat Pemeriksaan". Sistem kemudian mengambil data dari server dan menampilkannya dalam daftar kronologis. Pengguna dapat memilih salah satu riwayat untuk melihat detail lengkap hasil pemeriksaan dari dokter. Ilustrasi alur antarmuka untuk riwayat pemeriksaan dapat dilihat pada Gambar 2.6.

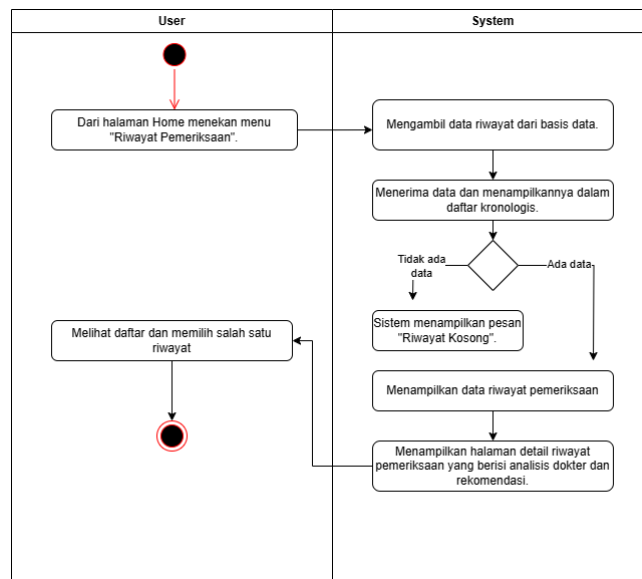
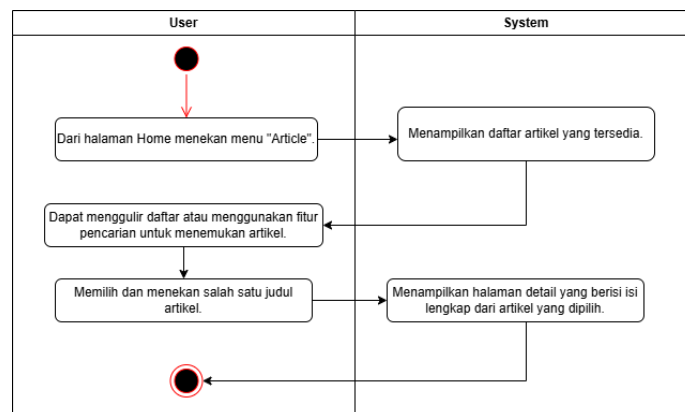
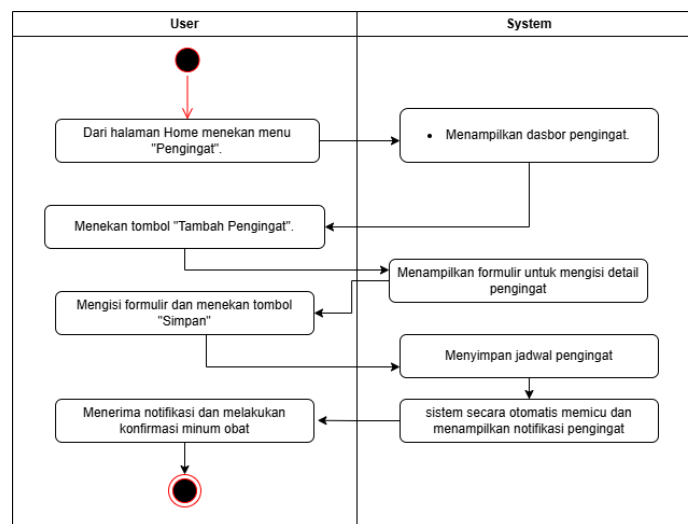
Gambar 2.6 *Activity Diagram* Riwayat Pemeriksaanf. *Activity Diagram* Artikel

Diagram ini menggambarkan alur pengguna saat mengakses, membaca, dan mencari artikel edukatif seputar TBC. Alur dimulai saat pengguna memilih menu "Artikel". Sistem menampilkan daftar artikel, di mana pengguna dapat melakukan *scroll* atau menggunakan fungsi pencarian. Setelah menemukan artikel yang diinginkan, pengguna menekannya untuk membaca isi secara detail. Alur antarmuka untuk fitur Artikel dapat dilihat pada Gambar 2.7.

Gambar 2.7 *Activity Diagram* Artikel

g. *Activity Diagram Peningat*

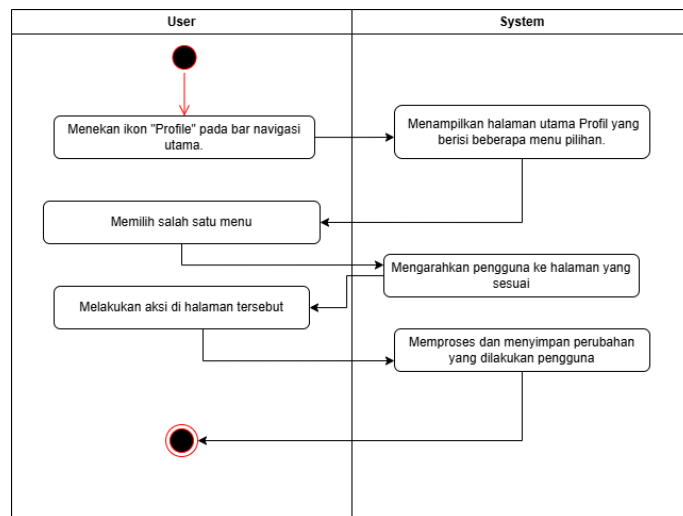
Diagram ini memodelkan proses dari pengaturan jadwal minum obat dan *check-up* oleh pengguna, pengiriman notifikasi oleh sistem, hingga konfirmasi minum obat. Alur dimulai saat pengguna menambah pengingat baru, mengisi detail, dan menyimpannya. Pada waktu yang ditentukan, sistem akan mengirimkan notifikasi. Alur antarmuka untuk fitur pengingat dapat dilihat pada Gambar 2.8.



Gambar 2.8 *Activity Diagram Peningat*

h. *Activity Diagram Profile*

Diagram ini menunjukkan bagaimana pengguna dapat melihat dan mengelola data profil serta pengaturan aplikasi. Alur dimulai saat pengguna memilih menu "Profil". Sistem akan menampilkan halaman utama profil yang berisi beberapa sub-menu. Pengguna dapat memilih menu "Edit Profil" untuk mengubah data diri, atau "Pengaturan" untuk mengubah kata sandi, tema tampilan, dan preferensi lainnya. Alur antarmuka untuk halaman profil dan pengaturan dapat dilihat pada Gambar 2.9.



Gambar 2.9 Activity Diagram Profile

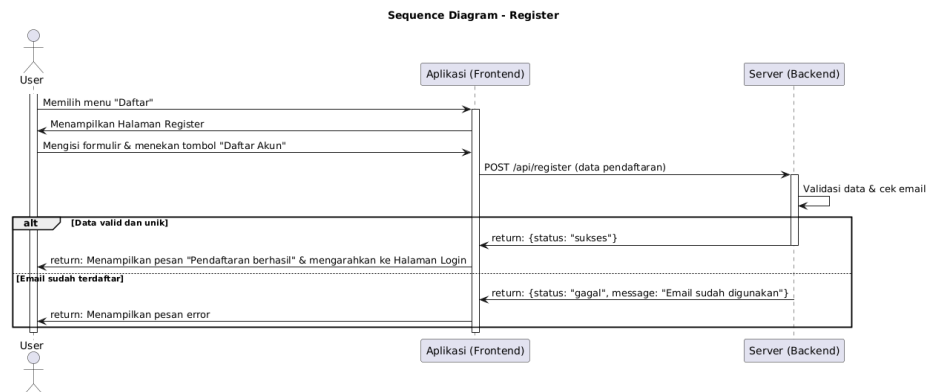
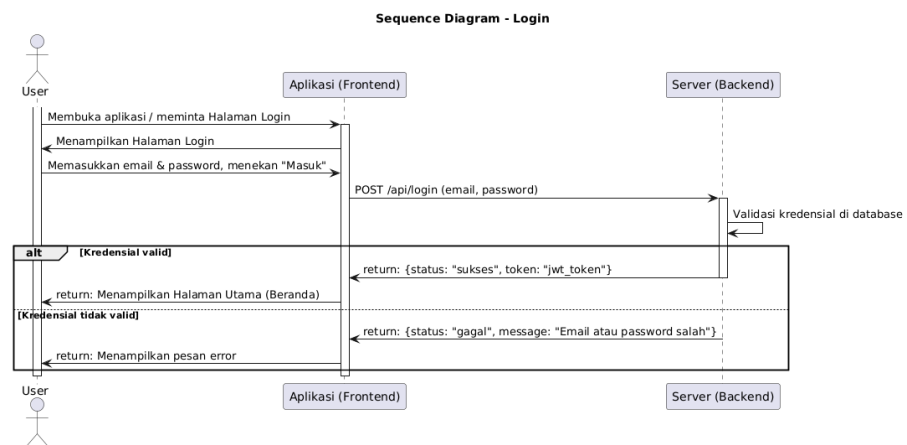
3. Sequence Diagram

Sequence Diagram atau Diagram Urutan adalah salah satu diagram UML yang digunakan untuk memodelkan interaksi antar objek dalam sistem berdasarkan urutan waktu. Diagram ini secara detail menggambarkan pesan atau perintah yang dikirimkan antara objek (*lifelines*)—seperti pengguna, antarmuka aplikasi, dan server untuk menyelesaikan sebuah fungsi atau *use case* [31].

Penjelasan berikut menyajikan urutan interaksi untuk setiap fitur dalam format teks terstruktur, dilengkapi dengan ilustrasi antarmuka pengguna sebagai konteks visual.

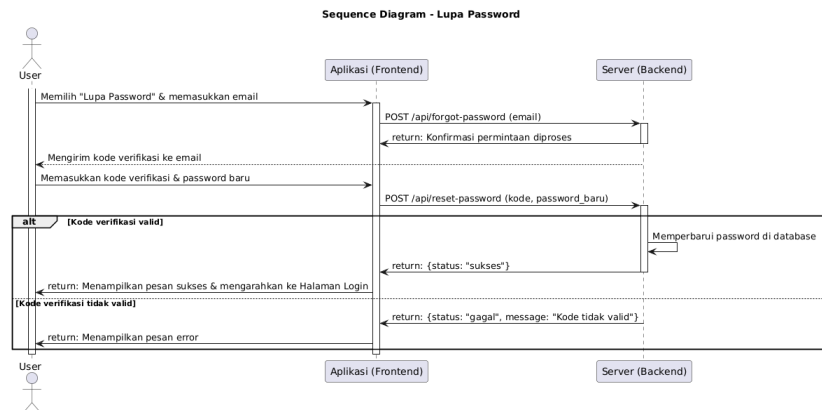
a. Sequence Diagram Register dan Login

Sequence Diagram ini menggambarkan proses autentikasi saat pengguna melakukan pendaftaran akun baru dan masuk ke dalam sistem. Alur interaksi untuk proses ini disajikan pada Gambar 2.10 dan Gambar 2.11.

Gambar 2.10 *Sequence Diagram Register*Gambar 2.11 *Sequence Diagram Login*

b. *Sequence Diagram Lupa Password*

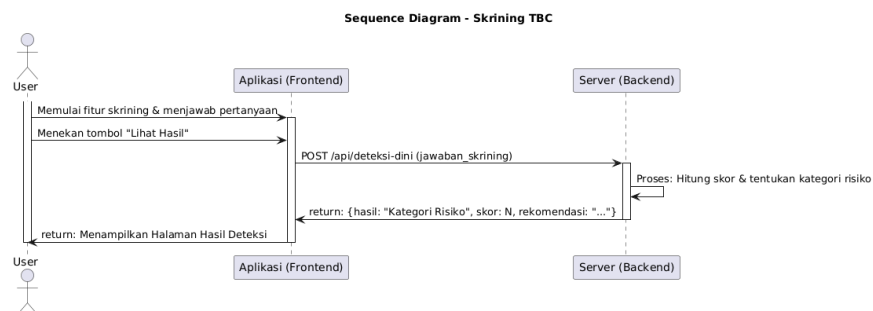
Sequence Diagram ini menggambarkan interaksi sistem saat pengguna meminta pengaturan ulang kata sandi karena lupa. Alur interaksi untuk proses ini disajikan pada Gambar 2.12.



Gambar 2.12 *Sequence Diagram* Lupa Password

c. *Sequence Diagram* Deteksi Dini/ Skrining TBC

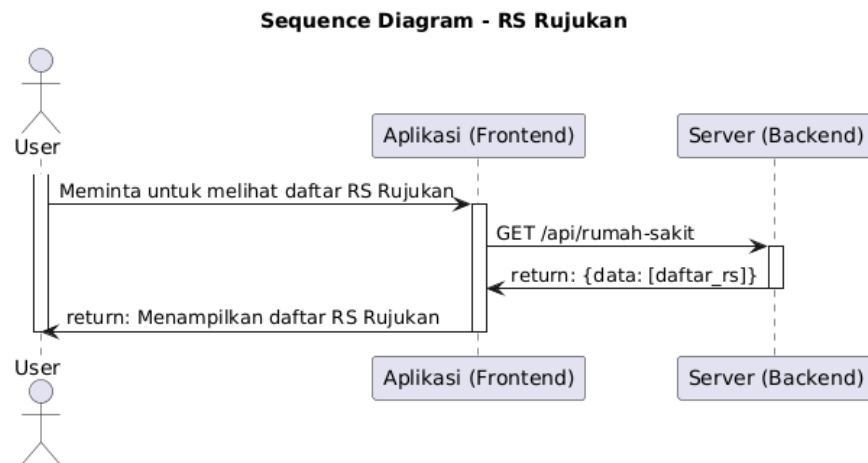
Sequence Diagram ini menggambarkan alur interaksi mulai dari pengguna menjawab kuesioner skrining hingga sistem memproses dan menampilkan hasil deteksi. Alur interaksi untuk proses ini disajikan pada Gambar 2.13.



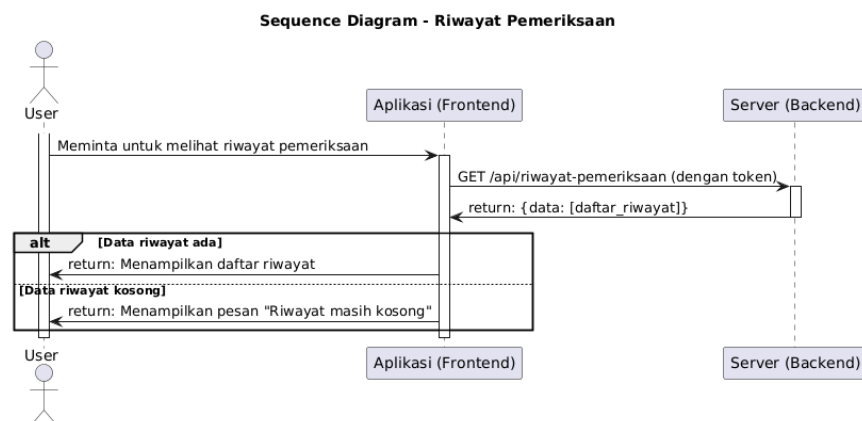
Gambar 2.13 *Sequence Diagram* Deteksi Dini/ Skrining TBC

d. *Sequence Diagram* RS Rujukan

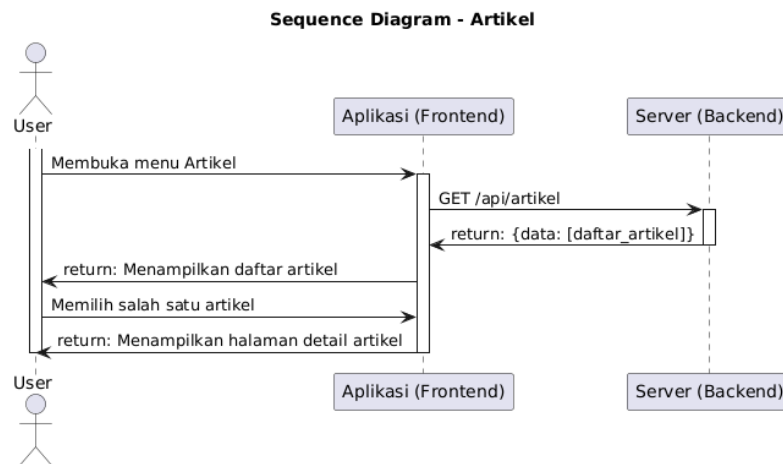
Sequence Diagram ini menggambarkan proses pengguna dalam mencari dan melihat informasi detail mengenai fasilitas kesehatan rujukan. Alur interaksi untuk proses ini disajikan pada Gambar 2.14.

Gambar 2.14 *Sequence Diagram* RS Rujukane. *Sequence Diagram* Riwayat Pemeriksaan

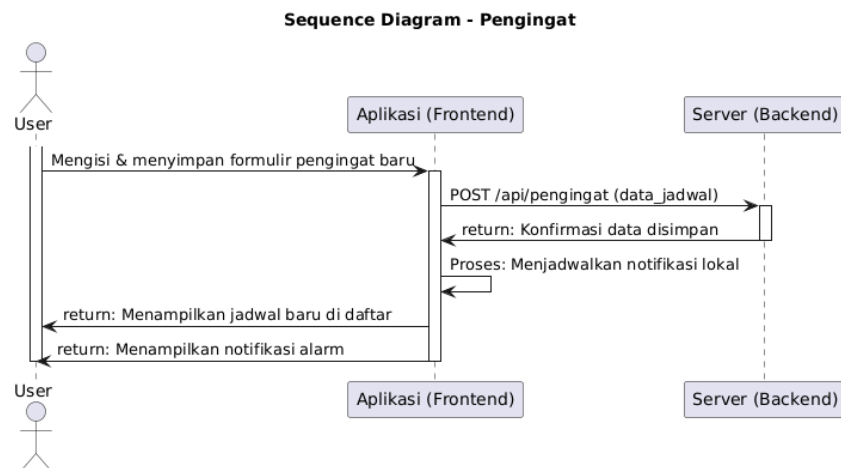
Sequence Diagram ini menggambarkan alur saat pengguna meminta dan menerima data riwayat pemeriksaan medis resmi dari sistem. Alur interaksi untuk proses ini disajikan pada Gambar 2.15.

Gambar 2.15 *Sequence Diagram* Riwayat Pemeriksaanf. *Sequence Diagram* Artikel

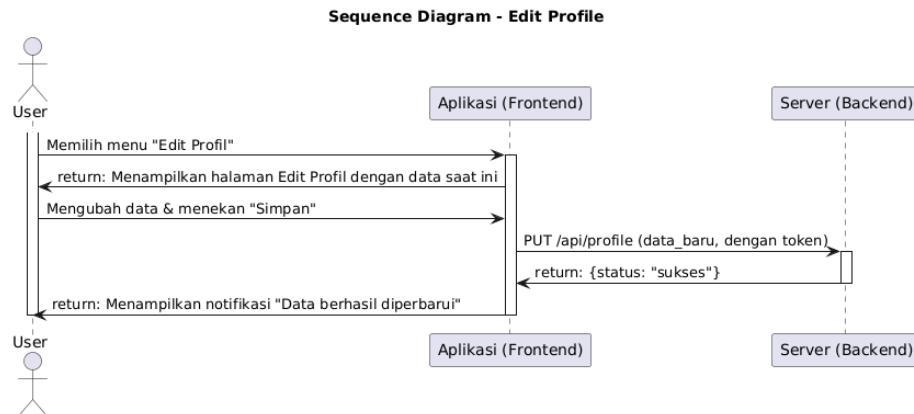
Sequence Diagram ini menggambarkan proses pengguna dalam mengakses daftar artikel dan membaca konten detailnya. Alur interaksi untuk proses ini disajikan pada Gambar 2.16.

Gambar 2.16 *Sequence Diagram* Artikelg. *Sequence Diagram* Pengingat

Sequence Diagram ini menggambarkan interaksi saat pengguna membuat jadwal pengingat baru dan proses sistem dalam mengirimkan notifikasi. Alur interaksi untuk proses ini disajikan pada Gambar 2.17.

Gambar 2.17 *Sequence Diagram* Pengingath. *Sequence Diagram* Profile

Sequence Diagram ini menggambarkan proses pengelolaan data profil dan pengaturan akun oleh pengguna, seperti saat melakukan pembaruan data. Alur interaksi untuk proses ini disajikan pada Gambar 2.18.



Gambar 2.18 *Sequence Diagram Profile*

4. Rancangan Desain Antarmuka (*Prototype*)

Desain antarmuka pengguna (*UI*) adalah aspek krusial yang menentukan bagaimana pengguna berinteraksi dengan aplikasi [30]. *UI* yang baik harus memastikan pengguna dapat menggunakan aplikasi dengan mudah dan nyaman. Untuk "RumahTBC", desain *UI* dibuat menggunakan Figma dengan fokus pada kemudahan penggunaan (*user-friendly*) dan tampilan yang informatif, tetapi tidak membingungkan pengguna awam.

Dalam penelitian ini, rancangan desain antarmuka digunakan untuk membuat prototipe visual aplikasi *mobile*. Prototipe ini berfungsi sebagai cetak biru yang menggambarkan secara detail setiap halaman dan komponen yang akan dibangun pada tahap implementasi. Tujuannya adalah untuk memvisualisasikan seluruh alur interaksi pengguna, mulai dari proses pendaftaran hingga penggunaan fitur-fitur inti aplikasi.

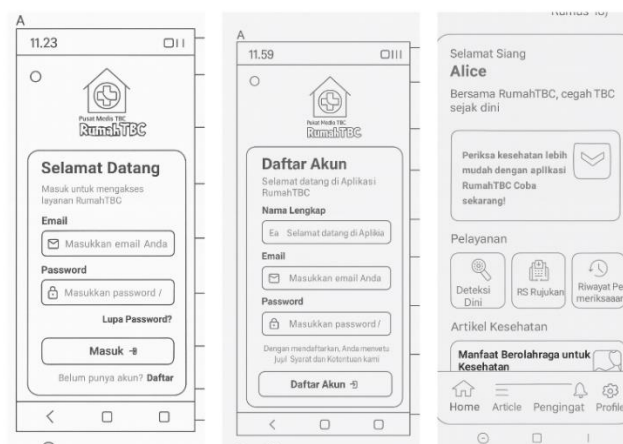
Berikut adalah rancangan desain antarmuka untuk setiap halaman fungsionalitas aplikasi "RumahTBC":

a. Rancangan Desain Antarmuka *Register* dan *Login*

Halaman ini merupakan gerbang utama bagi pengguna untuk mengakses aplikasi. Halaman *Register* dirancang untuk pengguna baru agar dapat membuat akun, sedangkan halaman *Login* digunakan oleh pengguna yang telah terdaftar untuk masuk ke dalam sistem. Pengguna baru mengisi formulir pendaftaran dan menekan tombol "Daftar" untuk membuat akun. Pengguna lama memasukkan kredensial mereka dan menekan tombol "Login" untuk mengakses halaman utama aplikasi. Sistem akan memvalidasi setiap data yang dimasukkan. Gambar 2.19 dan 2.20 menunjukkan desain antarmuka untuk proses autentikasi pengguna.



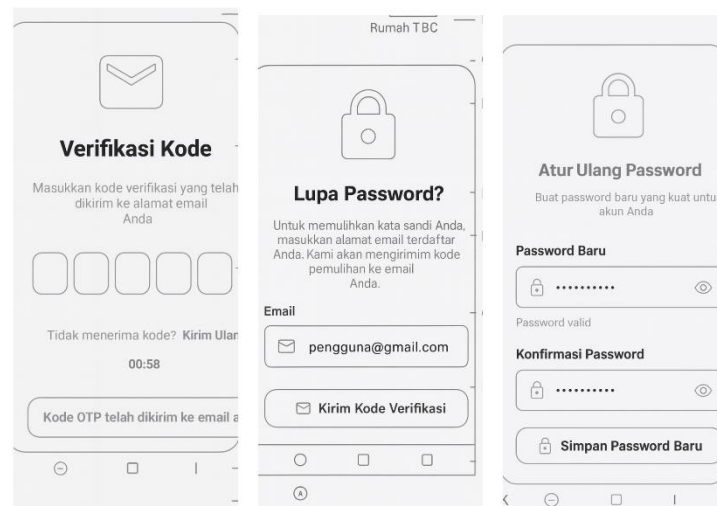
Gambar 2. 19 Rancangan Desain Antarmuka *Welcome*



Gambar 2. 20 Rancangan Desain Antarmuka *Login* dan *Register*

b. Rancangan Desain Antarmuka Lupa *Password*

Halaman ini berfungsi sebagai solusi bagi pengguna yang lupa dengan kata sandi akun mereka, sehingga mereka dapat memperoleh kembali akses ke aplikasi. Pengguna memasukkan alamat email mereka, kemudian sistem akan mengirimkan kode verifikasi untuk mengatur ulang kata sandi. Tampilan untuk fitur lupa *password* dapat dilihat pada Gambar 2.21.



Gambar 2. 21 Rancangan Desain Antarmuka Lupa *Password*

c. Rancangan Desain Antarmuka Deteksi Dini/ Skrining TBC

Halaman ini merupakan fitur inti aplikasi untuk melakukan deteksi dini tuberkulosis berdasarkan gejala klinis. Fitur ini bersifat informatif dan bertujuan untuk meningkatkan kesadaran pengguna akan potensi gejala TBC. Pengguna menjawab serangkaian pertanyaan mengenai gejala yang mereka alami. Setelah semua pertanyaan dijawab, pengguna menekan tombol "Lihat Hasil". Sistem kemudian akan memproses jawaban, menghitung skor, dan menampilkan hasil deteksi dalam dua kategori: Terindikasi Positif dan Negatif. Antarmuka untuk fitur skrining, mulai dari

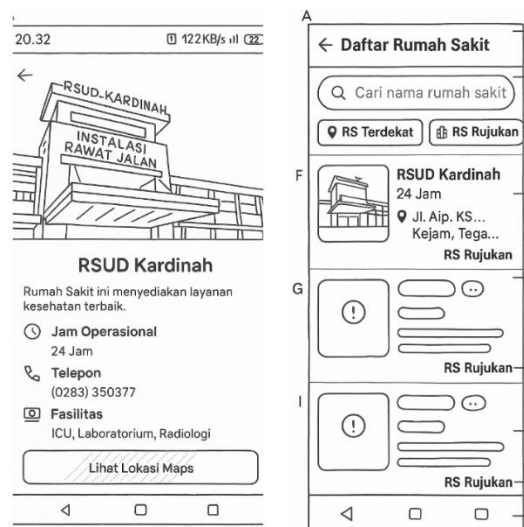
halaman informasi, proses menjawab pertanyaan, hingga penampilan hasil, diilustrasikan pada Gambar 2.22.



Gambar 2.22 Rancangan Desain Antarmuka Deteksi Dini/ Skrining TBC

d. Rancangan Desain Antarmuka Rumah Sakit Rujukan

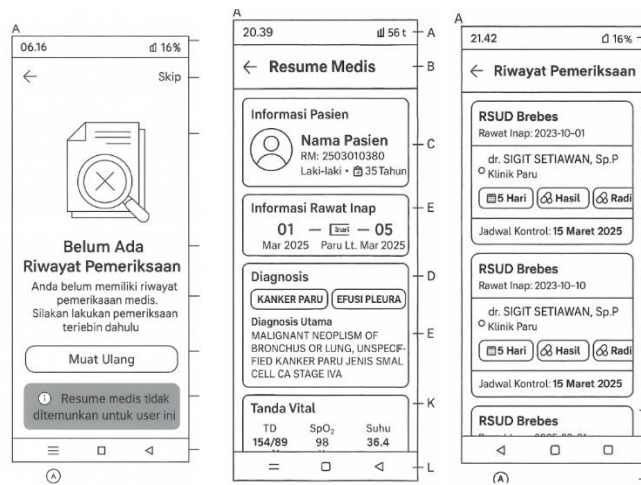
Menyediakan informasi mengenai fasilitas kesehatan terdekat yang dapat dikunjungi pengguna untuk pemeriksaan lebih lanjut, terutama jika hasil skrining menunjukkan adanya gejala yang perlu diwaspadai. Setelah mendapatkan hasil skrining teridentifikasi “*Positive*” pengguna akan direkomendasikan untuk mengunjungi halaman ini. Pengguna dapat melihat daftar fasilitas kesehatan dan memilih yang paling mudah dijangkau. Tampilan antarmuka untuk fitur RS Rujukan dapat dilihat pada Gambar 2.23.



Gambar 2.23 Rancangan Desain Antarmuka Rumah Sakit Rujukan

e. Rancangan Desain Antarmuka Riwayat Pemeriksaan

Halaman ini berfungsi sebagai rekam medis digital bagi pengguna, yang secara khusus menampilkan riwayat hasil pemeriksaan resmi dari fasilitas kesehatan (faskes) atau rumah sakit yang telah bekerja sama. Tujuannya adalah agar pengguna dapat memantau perkembangan kondisi kesehatannya secara kronologis berdasarkan data medis yang terverifikasi, bukan hasil skrining mandiri. Pengguna mengakses menu ini dari halaman *Home*. Data pada halaman ini tidak diinput manual oleh pengguna, melainkan muncul secara otomatis setelah pihak faskes mengirimkan hasil pemeriksaan ke sistem RumahTBC. Pengguna kemudian dapat melihat daftar riwayat dan detailnya kapan saja untuk memantau progres pengobatannya. Tampilan antarmuka untuk riwayat pemeriksaan dapat dilihat pada Gambar 2.24.



Gambar 2.24 Rancangan Desain Antarmuka Riwayat Pemeriksaan

f. Rancangan Desain Antarmuka Artikel

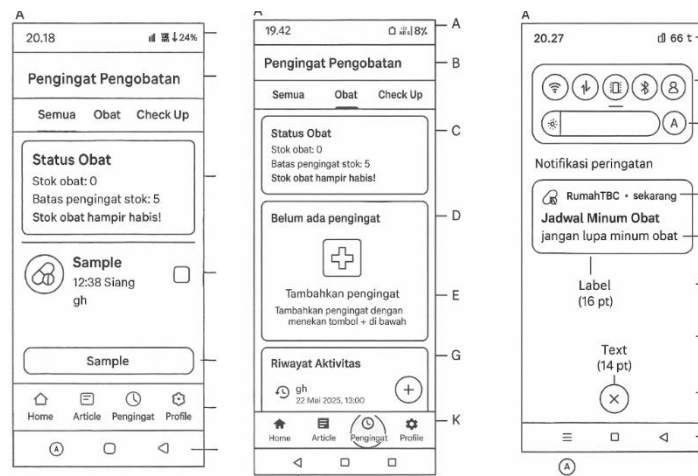
Berfungsi sebagai media edukasi untuk meningkatkan pengetahuan dan literasi masyarakat mengenai tuberkulosis, mulai dari pencegahan, gejala, hingga pengobatan. Pengguna dapat menjelajahi daftar artikel yang tersedia atau menggunakan fitur pencarian. Mereka dapat mengklik judul artikel untuk membaca isinya secara detail. Tampilan antarmuka untuk daftar artikel dan halaman detailnya dapat dilihat pada Gambar 2.25.



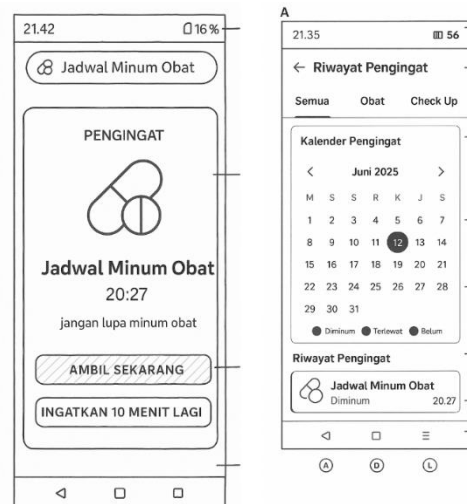
Gambar 2.25 Rancangan Desain Antarmuka Artikel

g. Rancangan Desain Antarmuka Pengingat

Fitur ini dirancang untuk membantu meningkatkan kepatuhan pengobatan pasien TBC. Halaman ini memungkinkan pengguna untuk mengatur dan melihat jadwal minum obat serta jadwal kontrol ke rumah sakit. Pengguna masuk ke halaman pengingat, lalu menekan tombol untuk menambah jadwal baru. Mereka akan mengisi detail seperti waktu minum obat dan menyimpannya. Aplikasi kemudian akan secara otomatis mengirimkan notifikasi sesuai jadwal yang telah diatur. Tampilan antarmuka untuk fitur pengingat dapat dilihat pada Gambar 2.26 dan 2.27.



Gambar 2. 26 Rancangan Desain Antarmuka Pengingat Awal



Gambar 2. 27 Rancangan Desain Antarmuka Pengingat Lanjutan

h. Rancangan Desain Antarmuka *Profile*

Halaman Profil Saya adalah pusat kendali bagi pengguna untuk mengelola seluruh aspek akun dan preferensi aplikasi mereka. Tujuannya adalah menyediakan akses terpusat untuk memperbarui data pribadi, mengatur keamanan akun, menyesuaikan pengalaman pengguna, dan menemukan informasi bantuan. Pengguna mengakses halaman ini melalui ikon "*Profile*" pada navigasi utama di bagian bawah layar. Dari sana, pengguna dapat memilih salah satu menu, misalnya "Edit Profil" untuk memperbarui nama atau nomor telepon, atau memilih "Pengaturan" untuk mengubah kata sandi, atur mode tampilan aplikasi, dan yang lainnya. Tampilan antarmuka untuk halaman profil dan menu pengaturan dapat dilihat pada Gambar 2.28 dan 2.29.



Gambar 2. 28 Rancangan Desain Antarmuka *Profile* Awal



Gambar 2. 29 Rancangan Desain Antarmuka *Profile* Lanjutan

5. Basis Data (*database*)

Basis data merupakan salah satu bagian penting dari penerapan teknologi informasi dan berfungsi sebagai pusat penyimpanan data yang akan diolah lebih lanjut [32]. Secara konseptual, basis data adalah kumpulan data yang saling berelasi sehingga dapat dimanipulasi, ditampilkan, dan dicari dengan cepat untuk mendukung tujuan dari sebuah sistem atau organisasi [32]. Perancangan basis data yang baik merupakan landasan utama untuk membangun sistem

informasi yang berhasil [32]. Desain struktur data ini dapat divisualisasikan menggunakan *Class Diagram*, yang merupakan inti dari desain berorientasi objek dan berfungsi memetakan struktur kelas, atribut, serta relasi antar objek yang akan diimplementasikan sebagai tabel dalam *database*.

- d. Teknologi dan Sistem: Aplikasi ini menggunakan sistem manajemen basis data relasional (MySQL). MySQL dipilih karena performanya yang tinggi, skalabilitas, serta kompatibilitasnya yang baik dengan *framework backend* Laravel.
- e. Struktur dan Tabel Utama: Struktur basis data dirancang untuk mendukung berbagai fitur inti aplikasi. Tabel-tabel utama yang saling terhubung ini meliputi:

- 1) Pasien: Menyimpan data profil dan kredensial pengguna aplikasi.

Tabel 2.1 Tabel *Database* Pasien

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap pengguna.
nama_lengkap	VARCHAR(55)		Nama lengkap pengguna.
email	VARCHAR(55)	UNIQUE	Alamat <i>email</i> untuk login (harus unik).
password	VARCHAR(128)		Kata sandi yang sudah di-hash.
nomor_telepon	VARCHAR(13)		Nomor telepon aktif pengguna.
nomor_pmo	VARCHAR(13)		Nomor WhatsApp PMO yang didaftarkan.
created_at	TIMESTAMP		Waktu pembuatan akun.

updated_at	TIMESTAMP		Waktu terakhir data diubah.
------------	-----------	--	-----------------------------

- 2) Deteksi_dini: Mencatat setiap sesi skrining yang dilakukan oleh pengguna, termasuk jawaban dan hasil skornya.

Tabel 2.2 Tabel *Database* Deteksi Dini

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap riwayat skrining.
user_id	INT	FK	Terhubung ke id pada tabel users.
skor_total	INT		Skor total hasil penjumlahan jawaban.
hasil	VARCHAR(255)		Hasil akhir (cth: 'Positif Skrining').
jawaban	JSON		Menyimpan detail jawaban kuesioner.
created_at	TIMESTAMP		Waktu skrining dilakukan.

- 3) Peningkat_obat: Mengelola jadwal minum obat yang dibuat oleh pengguna, termasuk waktu, dosis, dan status kepatuhan.

Tabel 2.3 Tabel *Database* Peningkat Obat

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap pengingat.
user_id	INT	FK	Terhubung ke id pada tabel users.
tipe_peningkat	ENUM('obat', 'kontrol')		Jenis pengingat.
judul	VARCHAR(48)		Nama obat atau nama kontrol.

waktu	TIME		Waktu pengingat akan aktif.
status	BOOLEAN		Status aktif atau tidak aktif.
created_at	TIMESTAMP		Waktu pengingat dibuat.

- 4) Artikel: Berisi konten edukatif seperti judul, isi, dan gambar artikel kesehatan yang akan ditampilkan di aplikasi.

Tabel 2.4 Tabel *Database* Artikel

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap artikel.
judul	VARCHAR(128)		Judul artikel.
isi_artikel	TEXT		Konten lengkap dari artikel.
gambar_url	VARCHAR(255)		URL gambar thumbnail artikel.
penulis	VARCHAR(55)		Nama penulis atau sumber artikel.
published_at	TIMESTAMP		Tanggal artikel dipublikasikan.

- 5) rumah_sakit: Menyimpan daftar fasilitas kesehatan rujukan beserta detailnya seperti alamat dan kontak.

Tabel 2.5 Tabel *Database* Rumah Sakit

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap rumah sakit.
nama_rs	VARCHAR(128)		Nama lengkap rumah sakit atau faskes.

alamat	TEXT		Alamat lengkap lokasi faskes.
nomor_telepon	VARCHAR(13)		Nomor telepon yang dapat dihubungi.
jam_operasional	VARCHAR(128)		Jam layanan (misal: "Senin - Jumat, 08:00 - 17:00").
latitude	DECIMAL(10, 8)		Koordinat latitude untuk fitur peta.
longitude	DECIMAL(11, 8)		Koordinat longitude untuk fitur peta.
gambar_url	VARCHAR(255)		URL gambar profil rumah sakit.
created_at	TIMESTAMP		Waktu data ditambahkan ke sistem.
updated_at	TIMESTAMP		Waktu terakhir data diubah.

- 6) Riwayat_pemeriksaan: Mencatat riwayat pemeriksaan resmi yang diinput oleh faskes, yang terhubung dengan data pasien.

Tabel 2.6 Tabel *Database* Riwayat Pemeriksaan

Nama Field	Tipe Data	Kunci	Keterangan
id	INT	PK	ID unik untuk setiap riwayat pemeriksaan.
user_id	INT	FK	Terhubung ke id pada tabel users (pasien).

rumah_sakit_id	INT	FK	Terhubung ke id pada tabel rumah_sakit.
tanggal_pemeriksaan	DATE		Tanggal saat pemeriksaan dilakukan.
diagnosis_dokter	TEXT		Hasil diagnosis resmi dari tenaga medis.
rekomendasi	TEXT		Saran atau rencana pengobatan dari dokter.
jadwal_kontrol	DATE		Tanggal yang ditetapkan untuk kontrol berikutnya.
diinput_oleh	VARCHAR(55)		Nama petugas faskes yang menginput data.
created_at	TIMESTAMP		Waktu data riwayat dibuat.
updated_at	TIMESTAMP		Waktu terakhir data riwayat diubah.

- f. Relasi dan Integritas Data: Tabel-tabel tersebut dihubungkan menggunakan relasi berbasis kunci asing (*foreign key*) untuk menjaga integritas data. Sebagai contoh, tabel deteksi_dini memiliki kolom pasien_id yang merujuk ke tabel pasien, memastikan bahwa setiap hasil skrining selalu terikat pada pengguna yang spesifik. Pendekatan relasional ini menjamin data tetap konsisten dan valid di seluruh bagian aplikasi.

2.1.2 Pembuatan Aplikasi

Tahap ini adalah proses implementasi dari seluruh rancangan yang telah dibuat sebelumnya diubah menjadi produk aplikasi yang fungsional. Proses ini melibatkan penerjemahan desain arsitektur, basis data, dan antarmuka menjadi kode program yang dapat dieksekusi, yang terbagi menjadi tiga bagian utama: pembuatan *backend*, pembuatan aplikasi *mobile*, dan pengujian.

1. Pembuatan *Backend* Aplikasi

Backend adalah bagian *server-side* dari aplikasi yang bertanggung jawab atas logika bisnis, pengelolaan data, dan komunikasi dengan basis data. Pada proyek "RumahTBC", *backend* berfungsi sebagai mesin utama yang mengolah semua permintaan dari aplikasi *mobile*.

a. Teknologi dan Arsitektur

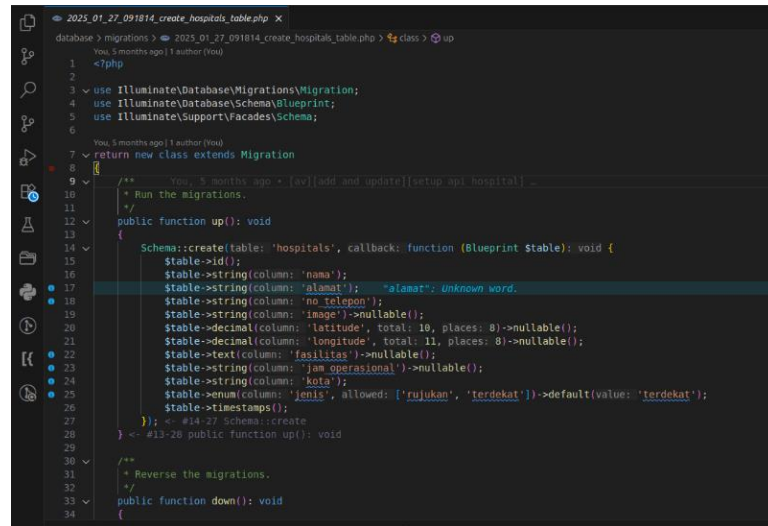
Pembuatan backend menggunakan *framework* Laravel yang berbasis bahasa pemrograman PHP. Arsitektur yang digunakan adalah *MVC (Model-View-Controller)*, yang memisahkan antara data (*Model*), logika (*Controller*), dan tampilan (*View*) untuk memastikan pengembangan yang terstruktur, aman, dan mudah dipelihara. Sistem basis data yang digunakan adalah MySQL.

b. Proses Implementasi

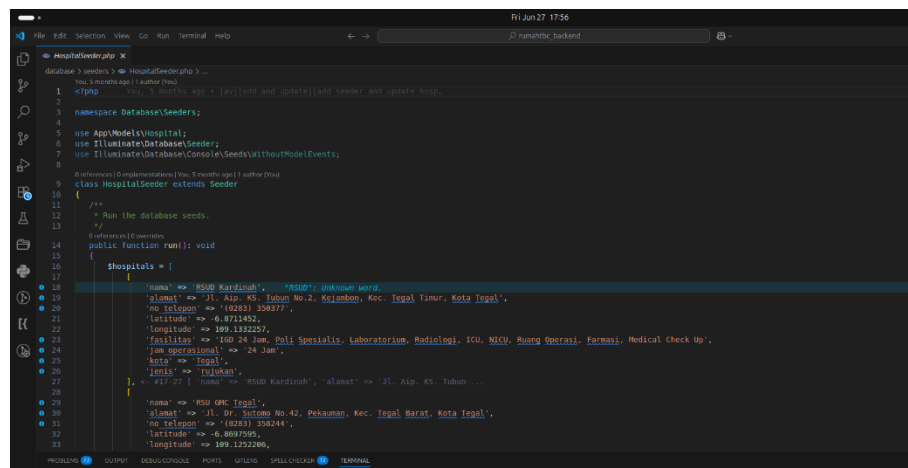
1) Migrasi dan *Seeder* Basis Data

Langkah awal dalam implementasi *backend* adalah mendefinisikan struktur basis data. Hal ini dilakukan menggunakan fitur *Migration* pada Laravel, yang memungkinkan pembuatan dan modifikasi skema tabel secara terprogram. Setelah struktur tabel terbentuk, *Seeder* digunakan

untuk mengisi data awal ke dalam *database*, seperti daftar rumah sakit rujukan atau akun admin default, untuk memastikan aplikasi memiliki data dasar saat pertama kali dijalankan. Contoh implementasi *migration* dan *seeder* bisa dilihat di Gambar 2.30 dan 2.31.



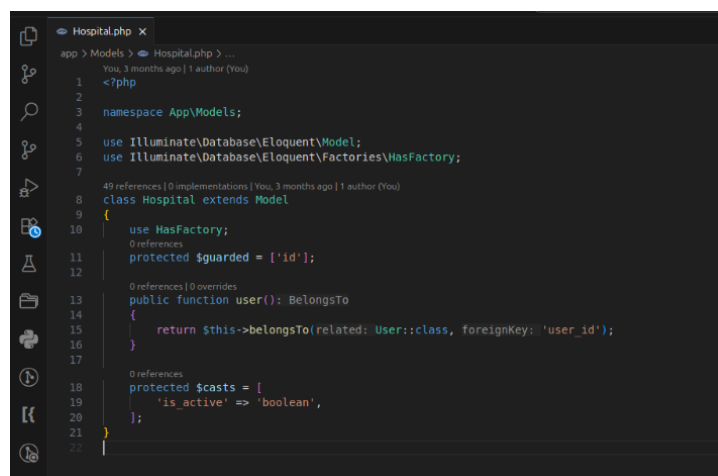
Gambar 2.30 Contoh Implementasi *Migration*



Gambar 2.31 Contoh Implementasi *Seeder*

2) Model (Eloquent ORM)

Setiap tabel dalam basis data direpresentasikan oleh sebuah kelas *Model* yang menggunakan *Eloquent ORM (Object-Relational Mapping)*. *Model* ini tidak hanya mendefinisikan atribut tabel, tetapi juga bertanggung jawab untuk semua interaksi dengan datanya, termasuk mendefinisikan relasi antar tabel. Implementasi dari *model* ini dapat dilihat pada Gambar 2.32.



```

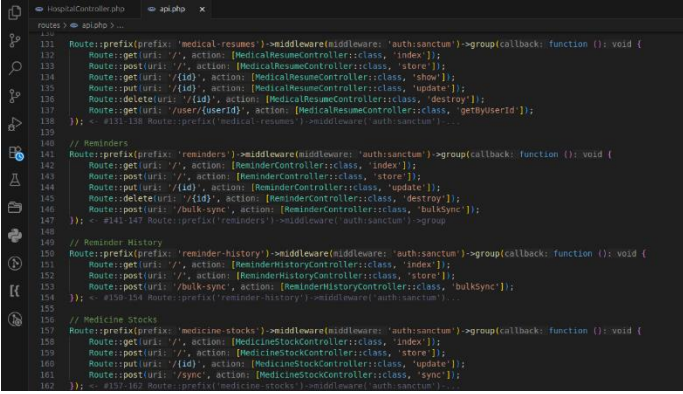
Hospital.php
app > Models > Hospital.php > ...
You, 3 months ago | 1 author (You)
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Model;
6 use Illuminate\Database\Eloquent\Factories\HasFactory;
7
8 class Hospital extends Model
9 {
10     use HasFactory;
11     protected $guarded = ['id'];
12
13     public function user(): BelongsTo
14     {
15         return $this->belongsTo(related: User::class, foreignKey: 'user_id');
16     }
17
18     protected $casts = [
19         'is_active' => 'boolean',
20     ];
21 }
22
  
```

Gambar 2. 32 Contoh Implementasi *Model*

3) Penyediaan RESTful API (*Route & Controller*)

Interaksi antara *backend* dan aplikasi *mobile* diatur melalui RESTful API. Proses ini melibatkan dua komponen utama:

- a) *Route*: Semua alamat URL atau *endpoint API* didefinisikan dalam file `routes/api.php`. *Route* ini berfungsi sebagai pemetaan yang mengarahkan setiap permintaan HTTP (misalnya, `GET /artikel`) ke fungsi yang sesuai di dalam *Controller*. Contoh implementasi *route* dapat dilihat pada Gambar 2.33.



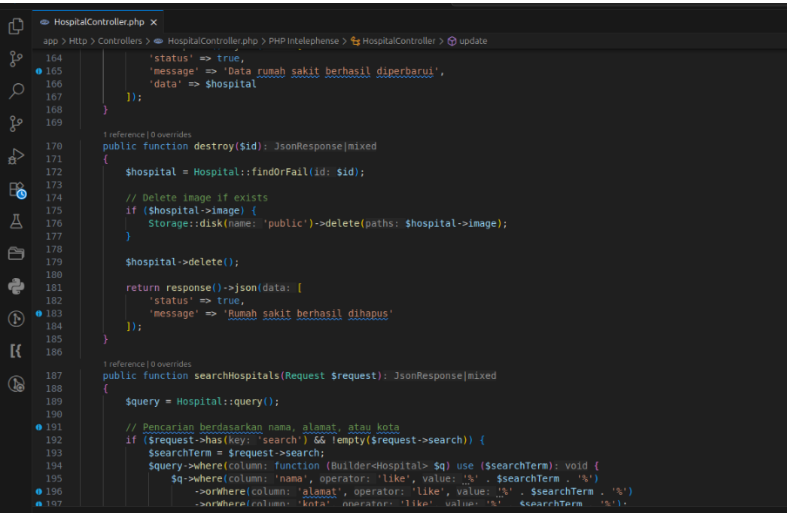
```

131 Route::prefix('medical-resumes')->middleware('auth:sanctum')->group(callback: function () { void {
132     Route::get('/', action: [MedicalResumeController::class, 'index']);
133     Route::post('/', action: [MedicalResumeController::class, 'store']);
134     Route::get('/:id', action: [MedicalResumeController::class, 'show']);
135     Route::put('/:id', action: [MedicalResumeController::class, 'update']);
136     Route::delete('/:id', action: [MedicalResumeController::class, 'destroy']);
137     Route::get('/user/{userId}', action: [MedicalResumeController::class, 'getByUserId']);
138 }); // #131-138 Route::prefix('medical-resumes')->middleware('auth:sanctum')->group(callback: function () { void {
139
140 // Reminders
141 Route::prefix('reminders')->middleware('auth:sanctum')->group(callback: function () { void {
142     Route::get('/', action: [ReminderController::class, 'index']);
143     Route::post('/', action: [ReminderController::class, 'store']);
144     Route::put('/:id', action: [ReminderController::class, 'update']);
145     Route::delete('/:id', action: [ReminderController::class, 'destroy']);
146     Route::post('/bulk-sync', action: [ReminderController::class, 'bulkSync']);
147 }); // #141-147 Route::prefix('reminders')->middleware('auth:sanctum')->group(callback: function () { void {
148
149 // Reminder History
150 Route::prefix('reminder-history')->middleware('auth:sanctum')->group(callback: function () { void {
151     Route::get('/', action: [ReminderHistoryController::class, 'index']);
152     Route::post('/', action: [ReminderHistoryController::class, 'store']);
153     Route::post('/bulk-sync', action: [ReminderHistoryController::class, 'bulkSync']);
154 }); // #150-154 Route::prefix('reminder-history')->middleware('auth:sanctum')->group(callback: function () { void {
155
156 // Medicine Stocks
157 Route::prefix('medicine-stocks')->middleware('auth:sanctum')->group(callback: function () { void {
158     Route::get('/', action: [MedicineStockController::class, 'index']);
159     Route::post('/', action: [MedicineStockController::class, 'store']);
160     Route::put('/:id', action: [MedicineStockController::class, 'update']);
161     Route::post('/sync', action: [MedicineStockController::class, 'sync']);
162 }); // #157-161 Route::prefix('medicine-stocks')->middleware('auth:sanctum')->group(callback: function () { void {

```

Gambar 2.33 Contoh Implementasi *Route API*

- b) **Controller**: Komponen ini berisi logika utama untuk menangani setiap permintaan yang masuk. Tugasnya meliputi validasi data, memanggil *Model* untuk operasi basis data, dan mengembalikan respons dalam format JSON ke aplikasi *mobile*. Contoh implementasi *controller* bisa dilihat pada Gambar 2.34.



```

164 'status' => true,
165 'message' => 'Data rumah sakit berhasil diperbarui',
166 'data' => $hospital
167 ];
168 }
169 }
170
171 public function destroy($id): JsonResponse|mixed
172 {
173     $hospital = Hospital::findOrFail($id);
174
175     // Delete image if exists
176     if ($hospital->image) {
177         Storage::disk('public')->delete($hospital->image);
178     }
179     $hospital->delete();
180
181     return response()->json(data: [
182         'status' => true,
183         'message' => 'Rumah sakit berhasil dihapus'
184     ]);
185 }
186
187 public function searchHospitals(Request $request): JsonResponse|mixed
188 {
189     $query = Hospital::query();
190
191     // Pencarian berdasarkan nama, alamat, atau kota
192     if ($request->hasKey('search') && !empty($request->search)) {
193         $searchTerm = $request->search;
194         $query->where(column: function (Builder $builder) use ($searchTerm) { void {
195             $builder->where(column: 'nama', operator: 'like', value: '%' . $searchTerm . '%')
196             ->orWhere(column: 'alamat', operator: 'like', value: '%' . $searchTerm . '%')
197             ->orWhere(column: 'kota', operator: 'like', value: '%' . $searchTerm . '%');

```

Gambar 2.34 Contoh Implementasi *Controller*

2. Pembuatan Aplikasi *Mobile*

Pembuatan aplikasi *mobile* adalah proses penerjemahan desain antarmuka (UI) dan pengalaman pengguna (UX) menjadi aplikasi *native* yang dapat diinstal dan dijalankan pada perangkat pengguna.

a. Teknologi dan Arsitektur

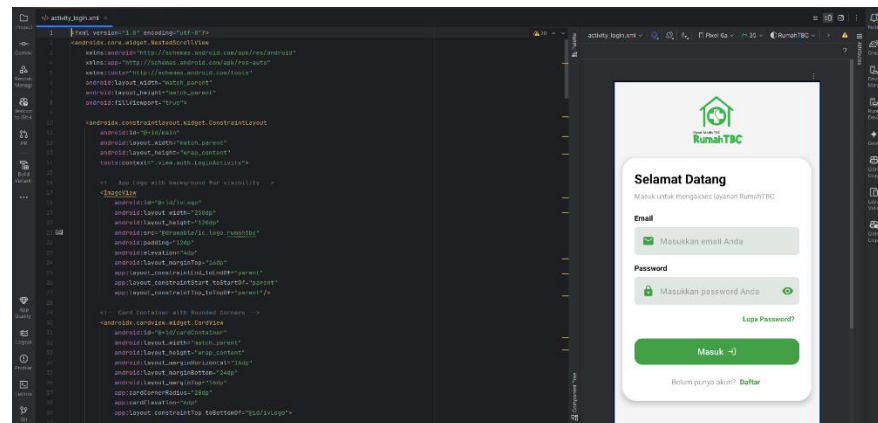
Aplikasi "RumahTBC" dikembangkan secara *native* untuk platform Android menggunakan bahasa pemrograman Kotlin. Arsitektur yang diadopsi adalah *MVVM (Model-View-ViewModel)*, sebuah pola desain modern yang direkomendasikan untuk pengembangan Android guna memisahkan logika antarmuka dari logika bisnis.

b. Proses Implementasi

1) *View (Layout XML)* dan *Activity*

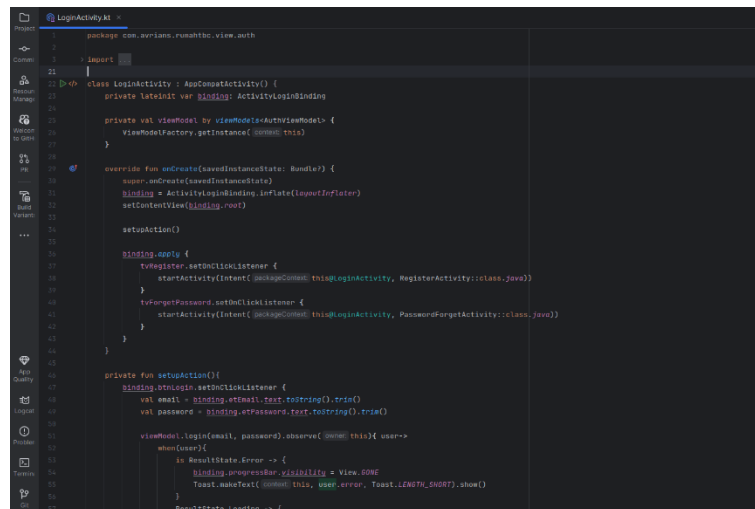
Dalam pengembangan aplikasi Android, *View* dan *Activity* adalah dua komponen yang bekerja sama untuk membentuk antarmuka pengguna (UI) dan mengelola interaksinya.

a) *View (Layout XML)*: Komponen ini bertanggung jawab untuk mendefinisikan struktur visual dan tata letak dari sebuah layar aplikasi. Desain antarmuka yang telah dibuat di Figma diterjemahkan menjadi kode dalam format XML. Setiap file XML (misalnya, `activity_login.xml`) berisi elemen-elemen UI seperti tombol (*Button*), kolom teks (*EditText*), dan gambar (*ImageView*). XML hanya mendefinisikan tampilan statis dari antarmuka. Contoh implementasi view seperti pada Gambar 2.35.



Gambar 2.35 Contoh Implementasi *View XML*

- b) *Activity (Kotlin)*: *Activity* adalah komponen dalam kode Kotlin yang memberikan "kehidupan" pada layout XML. Setiap *Activity* (misalnya, *LoginActivity.kt*) terhubung dengan satu file *layout XML*. Tugas utama *Activity* untuk memuat dan menampilkan *layout XML* ke layar. Selain itu, untuk mengelola logika di balik interaksi pengguna, seperti apa yang terjadi ketika sebuah tombol ditekan. Dan untuk berkomunikasi dengan komponen lain seperti *ViewModel* untuk meminta atau mengirim data. Sebagai contoh, *LoginActivity.kt* akan memuat *activity_login.xml*, kemudian menambahkan *onClickListener* pada tombol login untuk memicu proses autentikasi saat pengguna menekannya. Contoh implementasi *Activity* seperti Gambar 2.36.



```

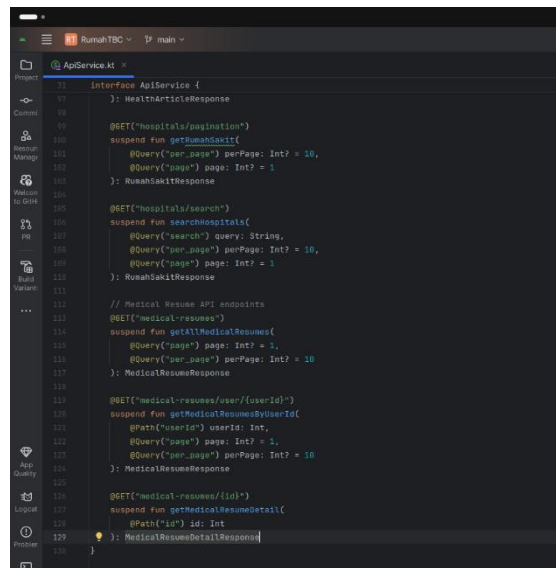
1 package com.sriani.rumahku.view.auth
2
3 import androidx.appcompat.app.AppCompatActivity
4 import androidx.databinding.BindingAdapter
5 import androidx.databinding.ViewDataBinding
6 import androidx.lifecycle.ViewModel
7 import androidx.lifecycle.ViewModelProvider
8 import androidx.lifecycle.ViewModelProviders
9 import androidx.lifecycle.ViewModelProvider.Factory
10 import androidx.lifecycle.ViewModelProvider.FactoryImpl
11 import androidx.lifecycle.ViewModelProvider.FactoryImpl
12 import androidx.lifecycle.ViewModelProvider.FactoryImpl
13 import androidx.lifecycle.ViewModelProvider.FactoryImpl
14 import androidx.lifecycle.ViewModelProvider.FactoryImpl
15 import androidx.lifecycle.ViewModelProvider.FactoryImpl
16 import androidx.lifecycle.ViewModelProvider.FactoryImpl
17 import androidx.lifecycle.ViewModelProvider.FactoryImpl
18 import androidx.lifecycle.ViewModelProvider.FactoryImpl
19 import androidx.lifecycle.ViewModelProvider.FactoryImpl
20 import androidx.lifecycle.ViewModelProvider.FactoryImpl
21
22 class LoginActivity : AppCompatActivity() {
23     private lateinit var binding: ActivityLoginBinding
24     private val viewModel by viewModel<AuthViewModel> {
25         viewModelFactory.getInstance()
26     }
27
28     override fun onCreate(savedInstanceState: Bundle?) {
29         super.onCreate(savedInstanceState)
30         binding = ActivityLoginBinding.inflate(layoutInflater)
31         setContentView(binding.root)
32         setupAction()
33
34         binding.register.setOnClickListener {
35             startActivity(Intent(this, RegisterActivity::class.java))
36         }
37         binding.forgetPassword.setOnClickListener {
38             startActivity(Intent(this, PasswordForgotActivity::class.java))
39         }
40     }
41
42     private fun setupAction() {
43         binding.login.setOnClickListener {
44             val email = binding.email.text.toString().trim()
45             val password = binding.password.text.toString().trim()
46             viewModel.login(email, password).observe(this) { user ->
47                 when(user){
48                     is Result.Success -> {
49                         binding.progressBar.visibility = View.GONE
50                         Toast.makeText(this, user.error, Toast.LENGTH_SHORT).show()
51                     }
52                     else -> {
53                         // Handle error
54                     }
55                 }
56             }
57         }
58     }
59 }

```

Gambar 2.36 Contoh Implementasi *Activity*

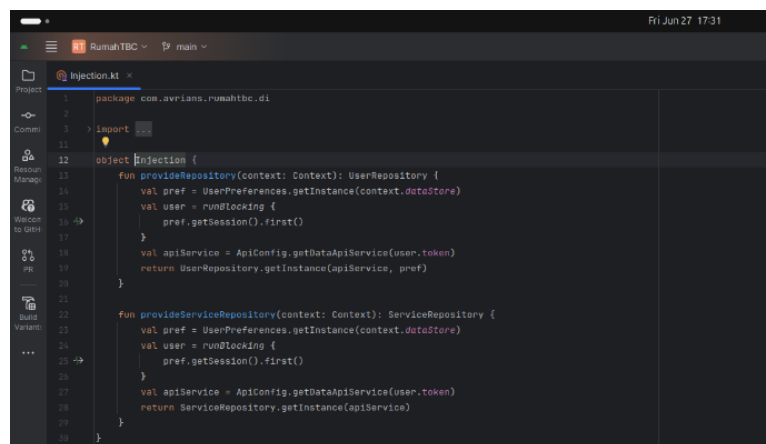
2) *API Service (Retrofit)*

Untuk berkomunikasi dengan *backend*, aplikasi *mobile* menggunakan *library* Retrofit. Langkah pertama adalah membuat sebuah *interface* di Kotlin yang mendefinisikan semua *endpoint* API yang akan diakses. *Interface* ini menggunakan anotasi dari Retrofit (seperti *@GET*, *@POST*, *@Body*, *@Header*) untuk menentukan metode HTTP, alamat URL, serta data yang akan dikirim. Retrofit kemudian akan secara otomatis mengimplementasikan *interface* ini untuk menangani komunikasi jaringan. Contoh implementasi *API Service* seperti pada Gambar 2.37.

Gambar 2.37 Contoh Implementasi *API Service*

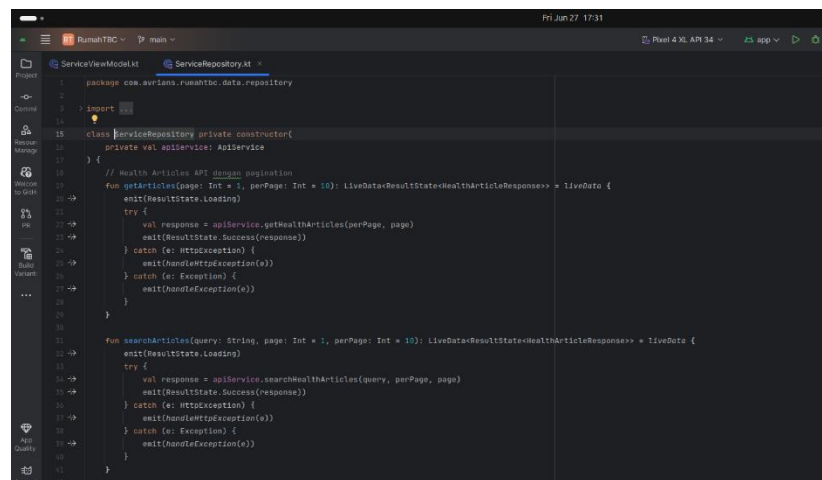
3) *Dependency Injection (DI)*

Aplikasi ini menerapkan prinsip *Dependency Injection* secara manual untuk menyediakan dependensi (objek yang dibutuhkan oleh objek lain). Tujuannya adalah untuk membuat kode lebih modular dan mudah diuji. Sebuah kelas *Injection* dibuat untuk menyediakan *instance* dari *Repository*. Contoh implementasi kelas *Injection* seperti Gambar 2.38.

Gambar 2.38 *Dependency Injection (DI)*

4) *Repository*

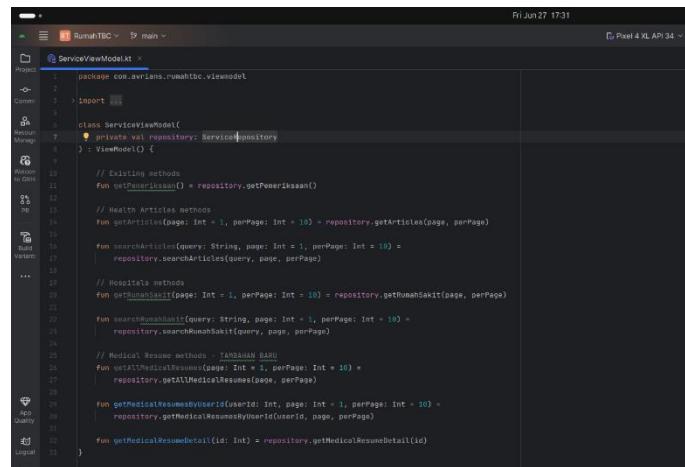
Repository berfungsi sebagai sumber data tunggal (*single source of truth*). Kelas ini mengelola logika pengambilan data, apakah data harus diambil dari jaringan melalui *API Service* atau dari *cache* lokal. Contoh implementasi *Repository* seperti pada Gambar 2.39.



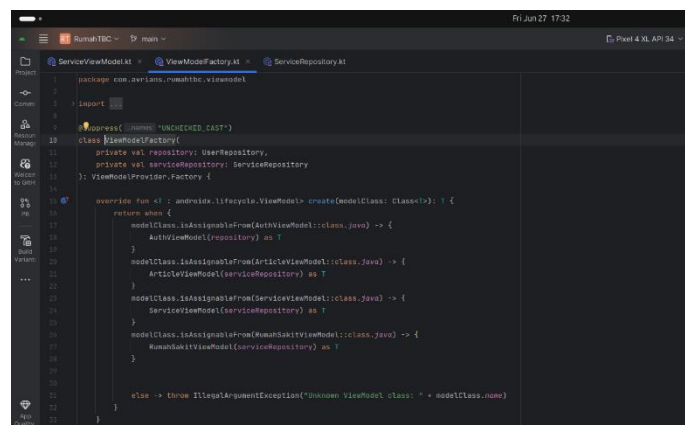
Gambar 2.39 Contoh implementasi *Repository*

5) *ViewModel* dan *ViewModelFactory*

a) *ViewModel*: Bertugas untuk menyimpan dan mengelola data yang berhubungan dengan UI. *ViewModel* mengambil data dari *Repository* dan menyajikannya ke *View* melalui objek *LiveData*. Contoh implementasi *ViewModel* seperti pada Gambar 2.40.

Gambar 2.40 Contoh implementasi *ViewModel*

b) *ViewModelFactory*: Digunakan untuk membuat *instance* dari *ViewModel*, terutama ketika *ViewModel* tersebut membutuhkan dependensi (seperti *Repository*) saat dibuat. Contoh implementasi *ViewModelFactory* seperti pada Gambar 2.41.

Gambar 2.41 Contoh implementasi *ViewModelFactory*

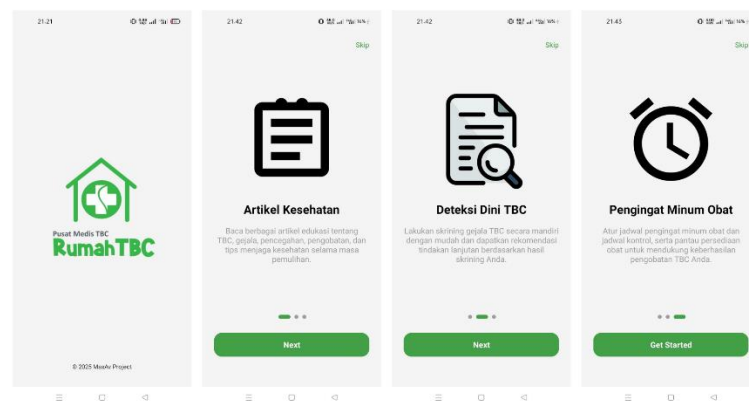
3. Tampilan Hasil Implementasi Aplikasi

Setelah proses perancangan dan pengkodean selesai, berikut adalah tampilan hasil akhir dari aplikasi "RumahTBC" yang telah berhasil diimplementasikan

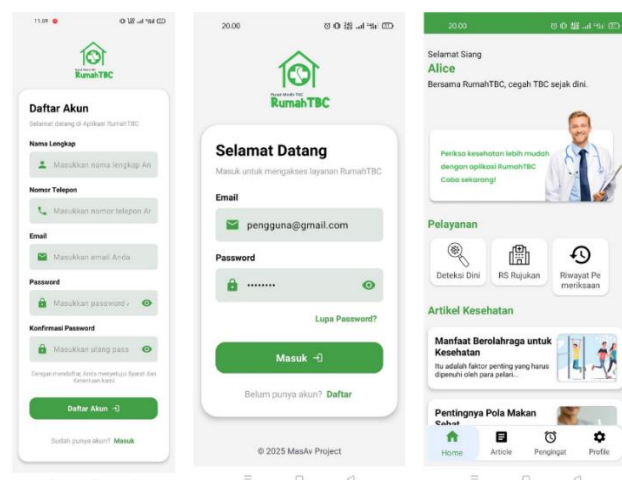
dan berjalan di perangkat Android. Tampilan ini merupakan wujud nyata dari prototipe desain yang telah dirancang sebelumnya.

a. Tampilan Halaman Welcome, Register, dan Login

Bagian ini menampilkan alur awal pengguna saat pertama kali membuka aplikasi. Dimulai dari halaman *welcome screen* yang memperkenalkan fitur utama, halaman pendaftaran (*register*) untuk pengguna baru, halaman *login* untuk pengguna terdaftar, hingga halaman utama (*home*) setelah berhasil masuk seperti pada Gambar 2.42 & 2.43.



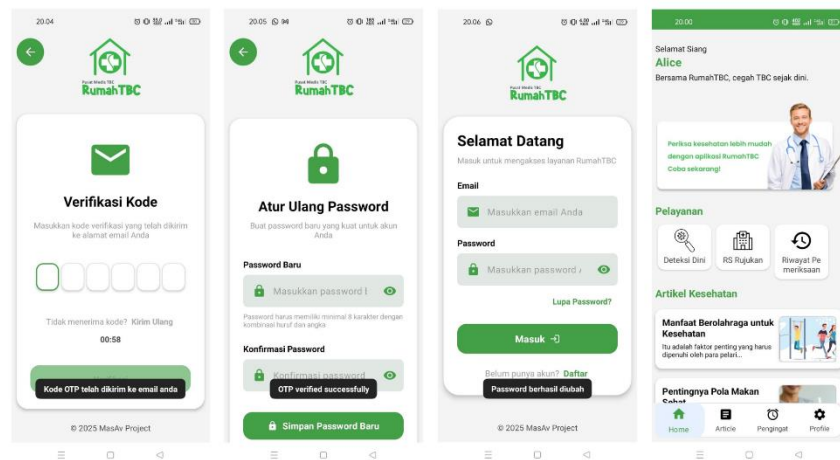
Gambar 2.42 Tampilan Halaman *Welcome*



Gambar 2.43 Tampilan Halaman Fitur *Login* dan *Register*

b. Tampilan Halaman Lupa Password

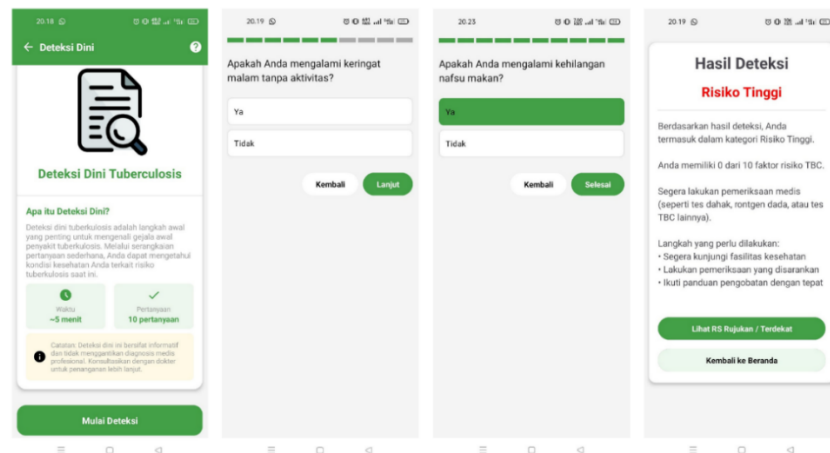
Berikut adalah alur yang dilalui pengguna ketika mereka lupa kata sandi. Prosesnya meliputi pengiriman kode verifikasi ke email pengguna, halaman untuk memasukkan kode tersebut, hingga halaman untuk mengatur ulang kata sandi baru, seperti pada Gambar 2.44.



Gambar 2.44 Tampilan Halaman Fitur Lupa Password

c. Tampilan Halaman Deteksi Dini/ Skrining TBC

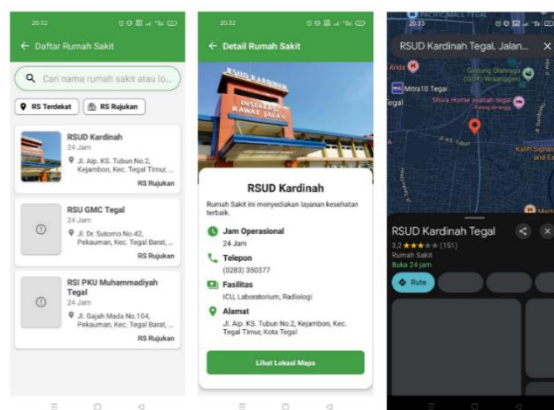
Ini adalah tampilan fitur inti skrining TBC. Pengguna memulai dari halaman pengantar, menjawab serangkaian pertanyaan mengenai gejala yang dialami, dan diakhiri dengan halaman hasil deteksi yang memberikan rekomendasi tindak lanjut jika terindikasi berisiko positif TBC, seperti pada Gambar 2.45.



Gambar 2.45 Tampilan Halaman Fitur Deteksi Dini/ Skrining TBC

d. Tampilan Halaman Rumah Sakit Rujukan

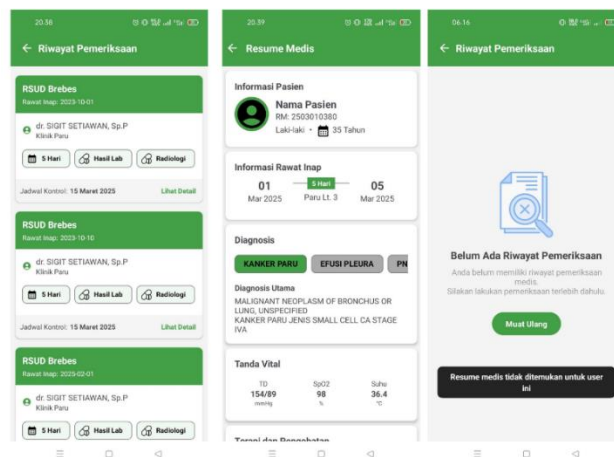
Setelah mendapatkan hasil skrining "*Positive*", pengguna dapat melihat daftar rumah sakit rujukan terdekat. Aplikasi juga menyediakan detail informasi seperti alamat dan jam operasional, integrasi dengan peta untuk menunjukkan Lokasi, serta bisa melakukan pendaftaran pemeriksaan seperti pada Gambar 2.46.



Gambar 2.46 Tampilan Halaman Fitur Rumah Sakit Rujukan

e. Tampilan Halaman Riwayat Pemeriksaan

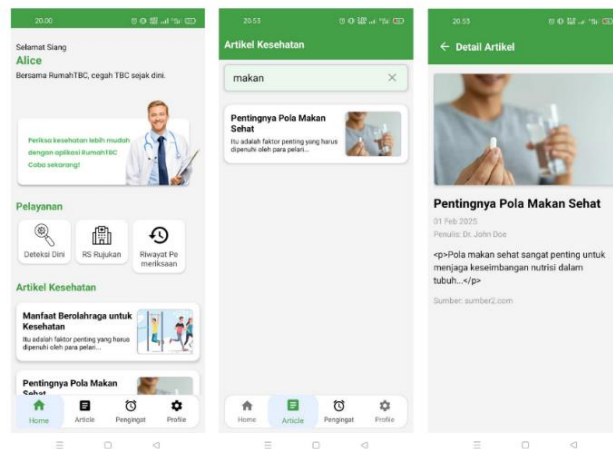
Halaman ini berfungsi sebagai rekam medis digital bagi pengguna. Tampilan sebelah kiri menunjukkan daftar riwayat pemeriksaan yang telah diinput oleh faskes. Tampilan tengah menunjukkan detail dari salah satu riwayat. Tampilan kanan adalah kondisi ketika pengguna belum memiliki data riwayat pemeriksaan, seperti pada Gambar 2.47.



Gambar 2.47 Tampilan Halaman Fitur Riwayat Pemeriksaan

f. Tampilan Halaman Artikel

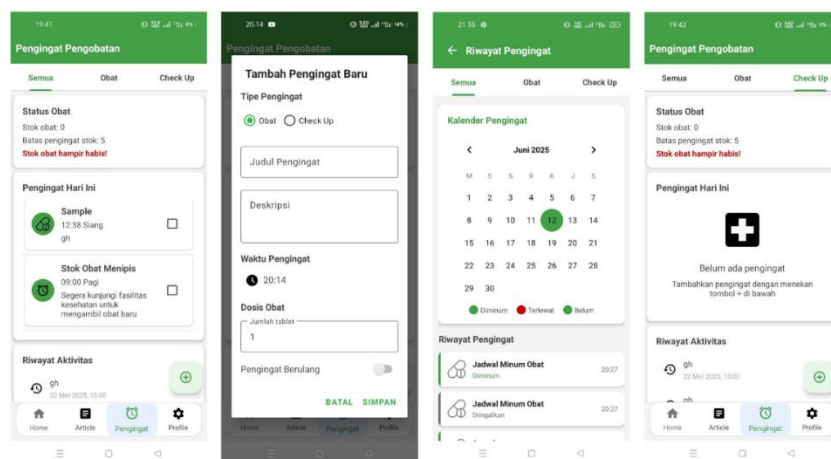
Fitur ini menyediakan konten edukasi seputar TBC. Pengguna dapat melihat daftar artikel di halaman utama, dan ketika salah satu artikel dipilih, aplikasi akan menampilkan halaman detail yang berisi konten lengkap dari artikel tersebut, seperti pada Gambar 2.48.



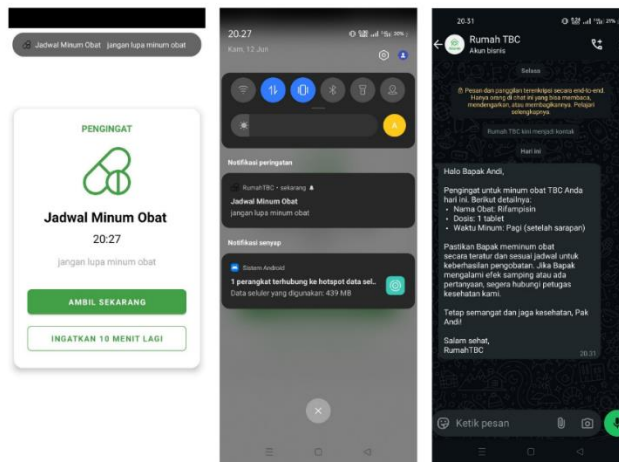
Gambar 2.48 Tampilan Halaman Fitur Artikel

g. Tampilan Halaman Peningkat

Bagian ini menampilkan alur lengkap fitur pengingat. Mulai dari halaman utama pengingat, formulir untuk menambah jadwal baru, notifikasi yang muncul di perangkat, hingga notifikasi pengingat yang dikirimkan melalui WhatsApp, seperti pada Gambar 2.49 dan 2.50.



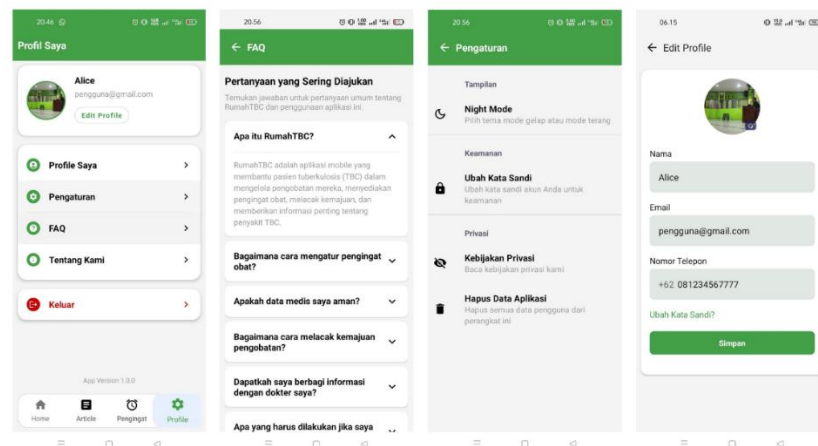
Gambar 2.49 Tampilan Halaman Fitur Peningkat Awal



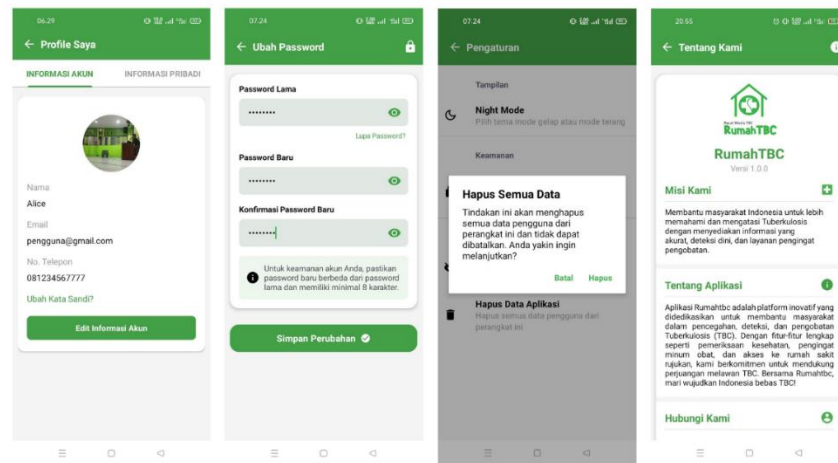
Gambar 2.50 Tampilan Halaman Fitur Pengingat Lanjutan

h. Tampilan Halaman Profile

Halaman profil menjadi pusat bagi pengguna untuk mengelola data akun mereka. Pengguna dapat melihat menu utama profil, mengedit informasi pribadi, mengubah kata sandi, hingga melihat informasi tentang aplikasi, seperti pada Gambar 2.51 dan 2.52.



Gambar 2.51 Tampilan Halaman Fitur *Profile*



Gambar 2.52 Tampilan Halaman Fitur Profile

4. Pengujian

a. Pengujian Fungsionalitas

Pengujian fungsional dilakukan dengan menggunakan metode *blackbox testing* untuk memastikan bahwa setiap fitur dalam aplikasi berjalan sesuai dengan fungsinya. Pengujian ini mencakup berbagai skenario guna memeriksa seluruh aspek aplikasi. Hasil pengujian disajikan pada Tabel 2.7.

Tabel 2.7 Tabel Pengujian Fungsionalitas

No	Skena rio Pengu jian	Test Case	Data Tes	Hasil yang di harapkan	Hasil Penguja n	Kesimpu lan
1	Auten tikasi: <i>Login</i> Sukse s	Pengguna memasukk an email dan kata sandi yang valid dan terdaftar, lalu menekan	Email: pengguna @gmail.co m & <i>Password:</i> password1 23	Sistem berhasil melakukan autentikasi dan mengarahka n pengguna ke halaman utama (beranda).		Berhasil

		tombol "Masuk".				
2	Autentikasi: <i>Login</i> Gagal	Pengguna memasukkan email yang benar tetapi dengan kata sandi yang salah.	Email: pengguna@gmail.com & Password: password345	Sistem menolak login dan menampilkan notifikasi "Email atau password salah".		Berhasil
3	Autentikasi: Registrasi Baru	Pengguna baru mengisi semua kolom pendaftaran dengan data yang valid dan unik, lalu menekan tombol "Daftar".	Nama: User Baru & Email: userbaru@gmail.com & Password: password123	Sistem berhasil membuat akun baru dan mengarahkan pengguna ke halaman login.		Berhasil
4	Autentikasi: Registrasi Gagal	Pengguna mencoba mendaftar menggunakan email yang sudah terdaftar di sistem	Email: pengguna@gmail.com	Sistem menolak pendaftaran dan menampilkan notifikasi bahwa email sudah digunakan.		Berhasil
5	Autentikasi: Lupa Password	Pengguna yang lupa kata sandi memasukkan email terdaftar dan mengikuti	Email: pengguna_valid@mail.com	Pengguna berhasil menerima kode verifikasi dan dapat mengatur kata sandi		Berhasil

		alur reset kata sandi hingga berhasil.		baru untuk login kembali.		
6	Skrining: Hasil Normal	Pengguna menjawab kuesioner skrining dengan jawaban yang menghasilkan skor rendah (0-3).	Jawaban yang tidak mengarah ke gejala TBC.	Sistem menampilkan halaman hasil "Risiko Rendah / Normal" tanpa rekomendasi ke RS Rujukan.		Berhasil
7	Skrining: Hasil Gejala Berat	Pengguna menjawab kuesioner skrining dengan jawaban yang menghasilkan skor tinggi (≥ 9).	Jawaban yang mengarah ke gejala TBC berat.	Sistem menampilkan hasil "Risiko Tinggi / Gejala Berat" dengan rekomendasi untuk segera ke faskes.		Berhasil
8	Fitur Informasi: RS Rujukan	Setelah mendapat hasil skrining risiko tinggi, pengguna menekan tombol untuk melihat daftar RS Rujukan.	-	Aplikasi menampilkan daftar fasilitas kesehatan terdekat beserta detailnya saat salah satunya dipilih.		Berhasil

9	Fitur Informasi: Artikel 1	Pengguna membuka menu "Artikel", memilih salah satu artikel dari daftar, dan melihat isinya.	-	Aplikasi berhasil menampilkan daftar artikel dan konten detail dari artikel yang dipilih.		Berhasil
10	Fitur Informasi: Riwayat Medis	Pengguna membuka menu "Riwayat Pemeriksaan" untuk melihat catatan medis resmi dari faskes.	-	Aplikasi menampilkan daftar riwayat pemeriksaan. (Asumsi: data sudah diinput oleh faskes).		Berhasil
11	Pengingat: Menambah Jadwal 1	Pengguna membuka fitur "Pengingat" dan berhasil menambahkan jadwal minum obat baru.	Waktu: 12:38	Jadwal baru berhasil tersimpan dan muncul di daftar pengingat.		Berhasil
12	Pengingat: Menerima Notifikasi	Sistem mengirimkan notifikasi push pada waktu yang telah dijadwalkan.	Jadwal dibuat pada jam 20:31.	Notifikasi pengingat muncul di perangkat pengguna pada jam 20:31		Berhasil

13	Profil: Edit Data Pengg una	Pengguna masuk ke menu "Profil Saya", memilih "Edit Profil", mengubah nomor telepon, lalu menyimpa n.	Nomor Telepon (baru): 08123456 7777	Perubahan data berhasil tersimpan dan ditampilkan pada halaman profil.		Berhasil
14	Profil: Ubah Kata Sandi	Pengguna masuk ke menu "Pengatura n" di dalam profil, lalu berhasil mengubah kata sandi lama ke kata sandi baru.	Password Lama: passwordB enar123 & Password Baru: passwordS angatBaru	Kata sandi berhasil diubah, dan pengguna harus menggunak an kata sandi baru untuk login berikutnya.		Berhasil
15	Profil: <i>Logou t</i>	Pengguna menekan tombol "Keluar" dari menu profil dan mengonfir masi.	-	Aplikasi berhasil mengeluark an pengguna dari sesi login dan kembali ke halaman login utama.		Berhasil

16	Fitur Informasi: Pencarian Artikel	Pengguna masuk ke menu "Artikel" dan menggunakan fitur pencarian untuk menemukan artikel dengan kata kunci tertentu.	Kata Kunci: "Pencegahan"	Aplikasi menampilkan daftar artikel yang relevan dengan kata kunci "Pencegahan".		Berhasil
17	Pengaturan: Mode Malam	Pengguna masuk ke menu "Pengaturan" dan mengaktifkan/menonaktifkan <i>Night Mode</i> .	-	Tampilan antarmuka aplikasi berubah antara tema terang dan gelap sesuai dengan pilihan pengguna.		Berhasil
18	Validasi Input: Form Kosong	Pengguna mencoba mendaftar akun baru tetapi tidak mengisi salah satu kolom wajib (misal: email).	Nama: User Test & Email: (kosong)	Sistem menolak pendaftaran dan menampilkan pesan peringatan untuk mengisi kolom yang kosong.		Berhasil
19	Navigasi: Halaman	Pengguna masuk ke menu "Profil Saya" dan	-	Aplikasi berhasil menampilkan halaman informasi		Berhasil

		menekan menu "FAQ" atau "Tentang Kami".		FAQ atau Tentang Kami yang sesuai.		
20	Penanganan Kondisi: Data Kosong	Pengguna baru yang belum memiliki riwayat medis membuka menu "Riwayat Pemeriksaan".	-	Aplikasi menampilkan halaman dengan pesan "Data riwayat pemeriksaan masih kosong" atau sejenisnya.		Berhasil

b. Pengujian *Usability*

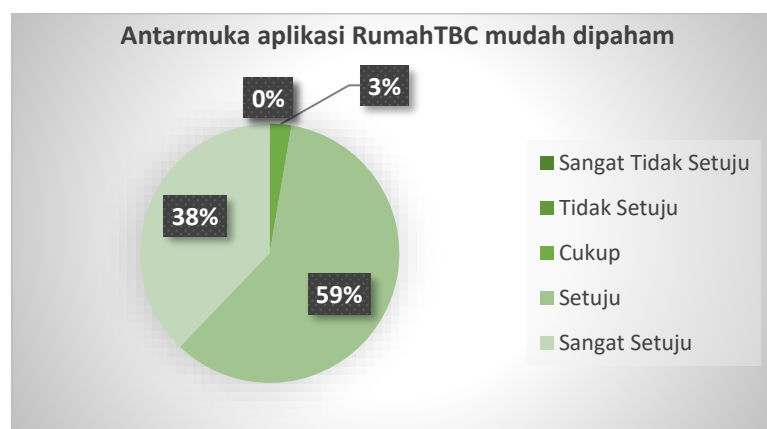
Selain pengujian fungsional, pengujian *usability* juga penting untuk mengukur tingkat kemudahan penggunaan dan kepuasan pengguna terhadap aplikasi. Meskipun tidak tercantum secara detail, pengujian ini dapat dilakukan dengan melibatkan calon pengguna untuk mencoba aplikasi dan memberikan masukan terkait pengalaman mereka dalam menggunakan fitur-fitur yang ada.

Pengujian usability melibatkan 37 responden yang mengisi kuesioner, dengan hasil yang divisualisasikan melalui diagram pie berikut:

1) Tampilan antarmuka aplikasi RumahTBC mudah dipahami?

Berdasarkan data yang telah diperoleh dari kuisoner, sebagian besar responden memberikan tanggapan positif, di mana sebanyak 59% responden menyatakan "Setuju" bahwa antarmuka aplikasi mudah

dipahami. Selain itu, 38% responden menyatakan "Sangat Setuju", yang menunjukkan tingkat kepuasan yang sangat tinggi terhadap tampilan dan kemudahan navigasi aplikasi. Sementara itu, hanya 3% responden yang menyatakan "Cukup", dan tidak ada responden yang memberikan tanggapan "Tidak Setuju" maupun "Sangat Tidak Setuju" (masing-masing 0%). Hasil ini dapat disimpulkan bahwa mayoritas pengguna merasa bahwa antarmuka aplikasi RumahTBC sudah dirancang dengan baik dan mudah dipahami, sehingga mendukung efektivitas penggunaan aplikasi dalam menyampaikan informasi dan layanan terkait tuberkulosis. Hasil dari data tersebut ditampilkan dalam diagram pie pada Gambar 2.53.

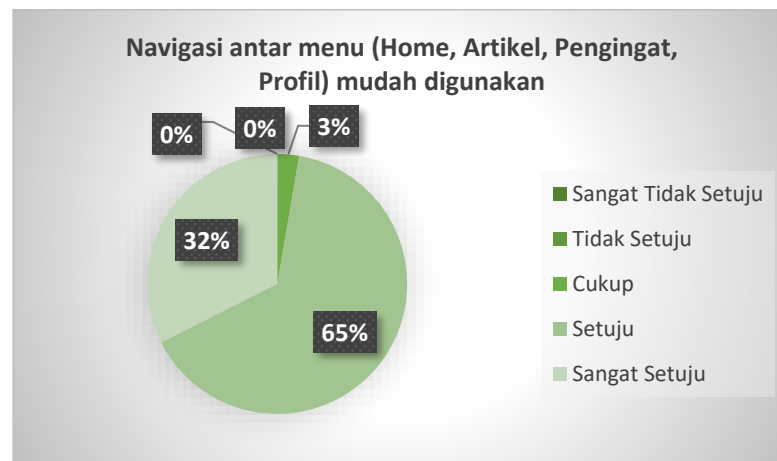


Gambar 2.53 Hasil Kuisisioner Antarmuka Mudah Dipahami

- 2) Navigasi antar menu (*Home*, Artikel, Pengingat, Profil) mudah digunakan?

Berdasarkan hasil evaluasi, mayoritas responden menyatakan bahwa navigasi antar menu pada aplikasi RumahTBC telah berjalan dengan sangat baik. Sebanyak 65% responden menyatakan setuju, dan 32%

lainnya menyatakan sangat setuju bahwa navigasi aplikasi mudah digunakan. selain itu, hanya 3% responden yang menilai navigasi berada pada kategori “cukup”, dan tidak ada responden yang menyatakan “tidak setuju” maupun “sangat tidak setuju”. Hasil ini memberikan kesimpulan bahwa struktur dan alur menu dalam aplikasi RumahTBC telah dirancang secara efektif dan ramah pengguna. Hasil dari tanggapan responden terhadap kemudahan navigasi antar menu disajikan dalam diagram pie pada Gambar 2.54.

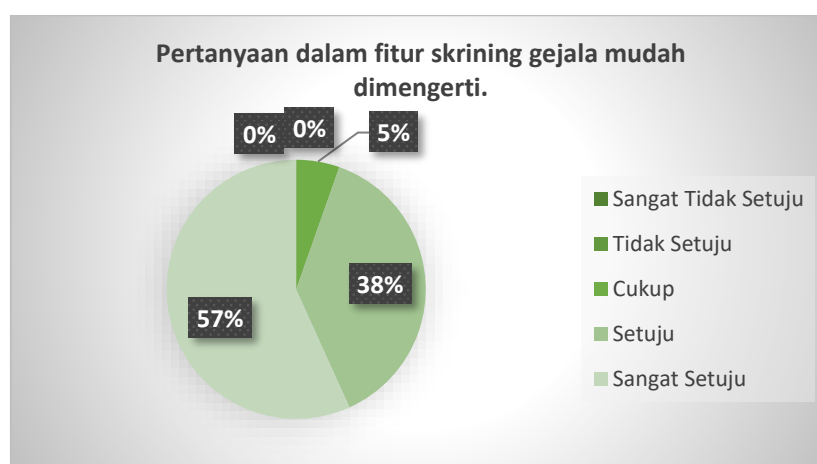


Gambar 2.54 Hasil Kuisisioner Navigasi Antar Menu

3) Pertanyaan dalam fitur skrining gejala mudah dimengerti?

Berdasarkan hasil evaluasi menunjukkan bahwa mayoritas responden merasa mudah memahami pertanyaan-pertanyaan yang terdapat dalam fitur skrining gejala pada aplikasi RumahTBC. Hal ini terbukti dari 57% responden yang menyatakan setuju dan 38% responden yang menyatakan sangat setuju terhadap kemudahan pemahaman isi pertanyaan. Sementara itu, hanya 5% responden yang menilai pertanyaan

tersebut dalam kategori “cukup”, dan tidak terdapat satupun responden yang memberikan penilaian “tidak setuju” maupun “sangat tidak setuju”. Hasil ini menunjukkan bahwa perancangan redaksi pertanyaan dalam fitur skrining telah sesuai dengan prinsip komunikasi kesehatan yang efektif dan inklusif. Ilustrasi hasil tanggapan responden tersebut disajikan pada diagram pie Gambar 2.55.

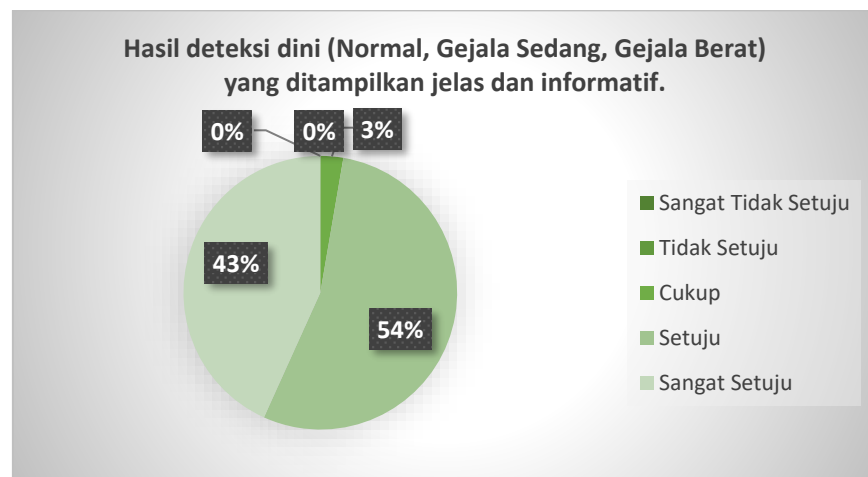


Gambar 2.55 Hasil Kuisisioner Fitur Skrining Gejala Mudah Dimengerti

- 4) Hasil deteksi dini (Normal, Gejala Sedang, Gejala Berat) yang ditampilkan jelas dan informatif?

Berdasarkan hasil kuisisioner, mayoritas menyatakan bahwa penyajian hasil deteksi dini pada aplikasi ini mudah dipahami dan informatif. Sebanyak 54% responden menyatakan setuju, dan 43% menyatakan sangat setuju terhadap pernyataan tersebut. Sementara itu, hanya 3% responden yang menilai cukup, dan tidak ada satu pun responden yang menyatakan tidak setuju maupun sangat tidak setuju. Hasil tersebut mencerminkan bahwa desain visual, bahasa, dan penjelasan yang

digunakan dalam menyampaikan hasil skrining telah sesuai dengan prinsip komunikasi yang efektif dalam layanan kesehatan digital. Rincian tanggapan responden dapat dilihat pada diagram pie Gambar 2.56.

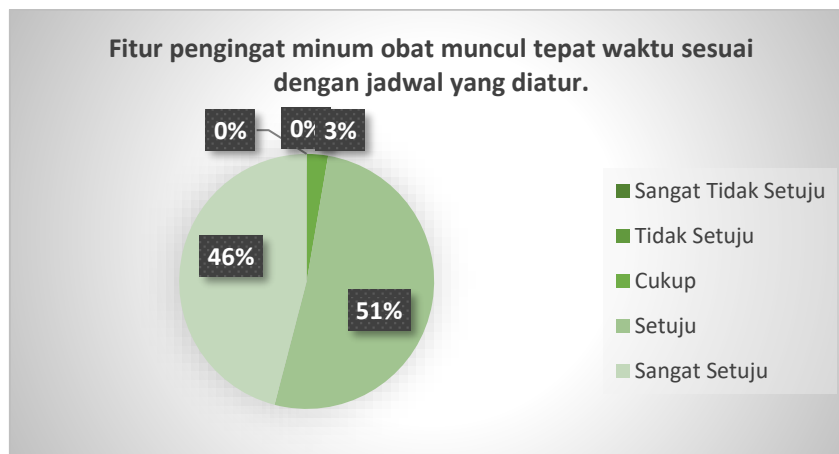


Gambar 2.56 Hasil Kuisioner Hasil Deteksi Dini

- 5) Fitur pengingat minum obat muncul tepat waktu sesuai dengan jadwal yang diatur?

Hasil kuisioner menunjukkan bahwa fitur pada RumahTBC berfungsi secara optimal dalam mendukung kepatuhan pengguna terhadap jadwal pengobatan. Sebanyak 51% responden menyatakan setuju bahwa pengingat muncul tepat waktu, dan 46% responden menyatakan sangat setuju. Hanya 3% responden yang memberikan penilaian cukup. Sementara itu, tidak ada responden yang menyatakan tidak setuju ataupun sangat tidak setuju. Hasil tersebut menunjukkan bahwa fitur pengingat dalam aplikasi RumahTBC telah berjalan efektif dan memenuhi harapan pengguna dalam membantu pengobatan yang teratur

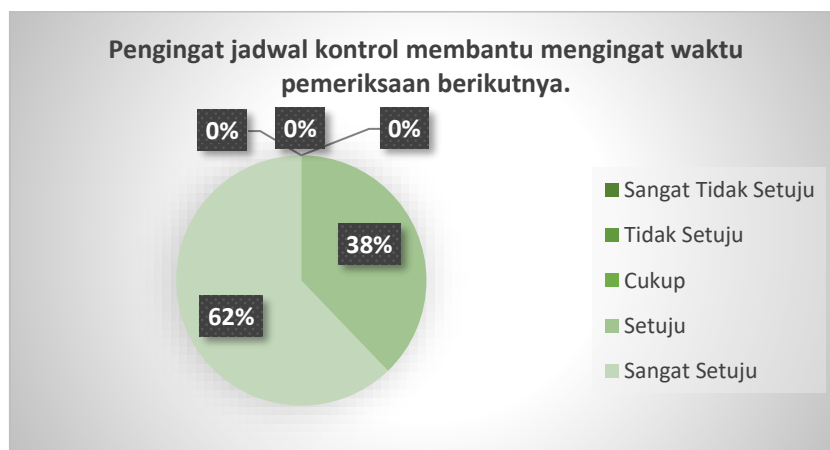
dan konsisten. Hasil tanggapan responden ditampilkan pada diagram pie Gambar 2.57.



Gambar 2.57 Hasil Kuisioner Fitur Pengingat Minum Obat

- 6) Pengingat jadwal kontrol membantu saya mengingat waktu pemeriksaan berikutnya?

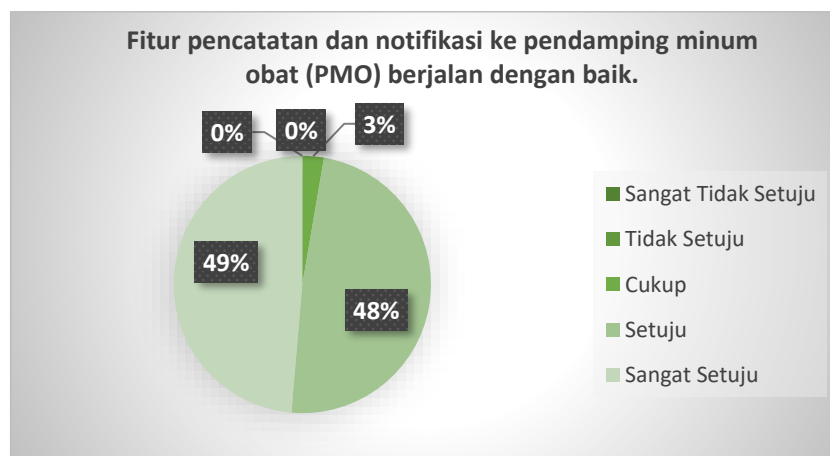
Berdasarkan hasil kuisioner, fitur ini dinilai sangat membantu oleh mayoritas pengguna. Sebanyak 62% responden menyatakan sangat setuju, dan 38% responden menyatakan setuju bahwa fitur pengingat kontrol memudahkan mereka dalam mengingat jadwal pemeriksaan yang telah ditetapkan. Sementara itu, tidak terdapat responden yang menyatakan tidak setuju, sangat tidak setuju, maupun menilai cukup. Hal ini membuktikan bahwa fitur pengingat jadwal kontrol telah bekerja secara optimal dan berhasil memenuhi kebutuhan pengguna. Rincian tanggapan pengguna terhadap fitur ini dapat dilihat melalui diagram pie Gambar 2.58.



Gambar 2.58 Hasil Kuisioner Pengingat Jadwal Kontrol

- 7) Fitur pencatatan dan notifikasi ke pendamping minum obat (PMO) berjalan dengan baik?

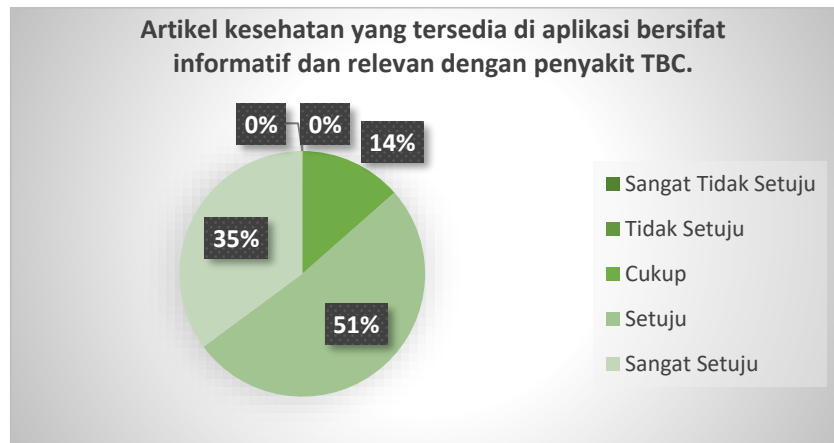
Hasil evaluasi kuisioner menunjukkan bahwa fitur ini telah berjalan dengan baik dan mendapat respons positif dari pengguna. Sebanyak 49% responden menyatakan sangat setuju, dan 48% responden menyatakan setuju bahwa fitur pencatatan dan notifikasi kepada PMO berfungsi secara efektif. Sementara itu, hanya 3% responden yang memberikan penilaian cukup, dan tidak terdapat responden yang menyatakan tidak setuju maupun sangat tidak setuju. Hasil ini memperkuat kesimpulan bahwa integrasi teknologi dalam sistem pendampingan pengobatan di aplikasi RumahTBC mampu memberikan dukungan nyata terhadap keberhasilan terapi pasien. Visualisasi tanggapan pengguna terhadap fitur ini dapat dilihat pada diagram pie Gambar 2.59.



Gambar 2.59 Hasil Kuisisioner Fitur Pencatatan dan Notifikasi

- 8) Artikel kesehatan yang tersedia di aplikasi bersifat informatif dan relevan dengan penyakit TBC?

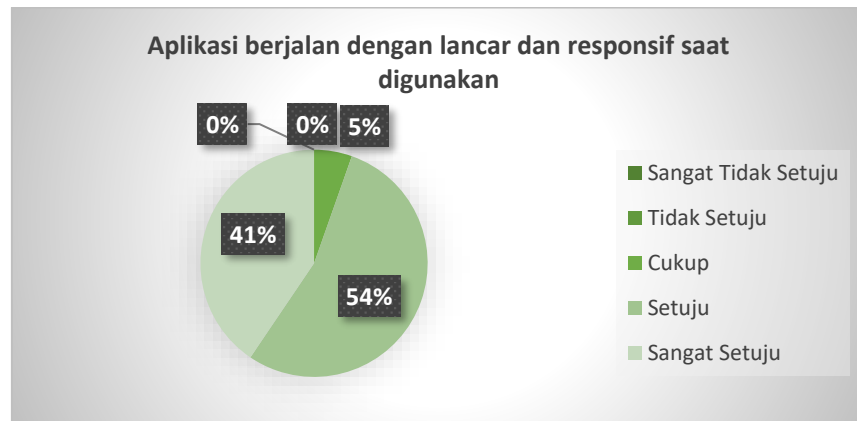
Berdasarkan hasil evaluasi kuisisioner, sebagian besar pengguna menilai bahwa artikel kesehatan yang disediakan dalam aplikasi ini informatif dan relevan dengan isu TBC. Hal ini tercermin dari 51% responden yang menyatakan setuju dan 35% responden yang menyatakan sangat setuju terhadap pernyataan tersebut. Sebanyak 14% responden memberikan penilaian cukup, sementara tidak ada satu pun responden yang menyatakan tidak setuju ataupun sangat tidak setuju. Data ini menunjukkan bahwa konten artikel yang disajikan telah memenuhi harapan mayoritas pengguna dalam hal kejelasan informasi dan kesesuaian topik. Hasil tanggapan responden terhadap kualitas artikel kesehatan dalam aplikasi RumahTBC disajikan melalui diagram pie Gambar 2.60.



Gambar 2.60 Hasil Kuisisioner Artikel Kesehatan

9) Aplikasi berjalan dengan lancar dan responsif saat digunakan?

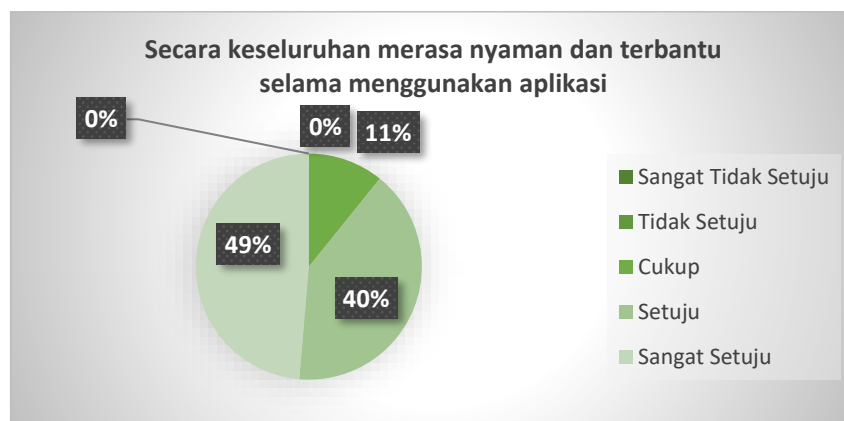
Berdasarkan hasil evaluasi kuisisioner, aplikasi RumahTBC dinilai memiliki performa yang baik. Sebanyak 54% responden menyatakan setuju, dan 41% menyatakan sangat setuju bahwa aplikasi berjalan dengan lancar dan responsif saat digunakan. Hanya 5% responden yang memberikan penilaian cukup, dan tidak ada responden yang menyatakan tidak setuju maupun sangat tidak setuju. Hasil ini mengindikasikan bahwa secara teknis, aplikasi telah berhasil memenuhi standar kenyamanan dalam penggunaan, serta menunjukkan stabilitas sistem yang mendukung fungsionalitasnya. Tampilan visual dari hasil tanggapan responden disajikan melalui diagram pie Gambar 2.61.



Gambar 2.61 Hasil Kuisisioner Aplikasi Lancar dan Responsif

10) Secara keseluruhan, saya merasa nyaman dan terbantu selama menggunakan aplikasi ini?

Berdasarkan hasil tanggapan pengguna terhadap aplikasi RumahTBC, mayoritas responden menyatakan bahwa mereka merasa nyaman dan terbantu selama menggunakan aplikasi ini. Sebanyak 49% responden menyatakan sangat setuju, dan 40% responden menyatakan setuju dengan pernyataan tersebut. Sebagian kecil, yaitu 11% responden, menilai cukup. Tidak terdapat responden yang menyatakan tidak setuju ataupun sangat tidak setuju. Hasil tersebut mencerminkan bahwa secara keseluruhan, aplikasi RumahTBC telah mampu memberikan pengalaman pengguna yang positif, baik dari sisi fungsionalitas, tampilan, maupun efektivitas fitur yang disediakan. Diagram pie Gambar 2.62.



Gambar 2.62 Hasil Kuisioner Secara Keseluruhan

Dari data yang diperoleh, berikut hasil perhitungan pengujian usability yang diperoleh dari 37 responden sesuai pada Tabel 2.8.

Tabel 2.8 Tabel Hasil Perhitungan Pengujian *Usability*

No	Aspek yang Dinilai	SS (×5)	S (×4)	C (×3)	TS (×2)	STS (×1)	Total Skor
1	Antarmuka mudah dipahami	14×5 = 70	22×4 = 88	1×3 = 3	0×2 = 0	0×1 = 0	161
2	Navigasi antar menu mudah digunakan	12×5 = 60	24×4 = 96	1×3 = 3	0×2 = 0	0×1 = 0	159
3	Pertanyaan skrining mudah dimengerti	14×5 = 70	21×4 = 84	2×3 = 6	0×2 = 0	0×1 = 0	160
4	Hasil deteksi dini jelas dan informatif	16×5 = 80	20×4 = 80	1×3 = 3	0×2 = 0	0×1 = 0	163
5	Pengingat minum obat tepat waktu	17×5 = 85	19×4 = 76	1×3 = 3	0×2 = 0	0×1 = 0	164
6	Pengingat jadwal kontrol membantu	23×5 = 115	14×4 = 56	0×3 = 0	0×2 = 0	0×1 = 0	171
7	Pencatatan & notifikasi ke PMO baik	18×5 = 90	18×4 = 72	1×3 = 3	0×2 = 0	0×1 = 0	165

8	Artikel informatif & relevan dengan penyakit TBC	13×5 = 65	19×4 = 76	5×3 = 15	0×2 = 0	0×1 = 0	156
9	Aplikasi berjalan lancar dan responsif	15×5 = 75	20×4 = 80	2×3 = 6	0×2 = 0	0×1 = 0	161
10	Merasa nyaman dan terbantu	18×5 = 90	15×4 = 60	4×3 = 12	0×2 = 0	0×1 = 0	162
Total							1.622

Evaluasi dilakukan terhadap 10 aspek usability aplikasi RumahTBC oleh 37 responden. Dalam pengujian ini, pengguna diminta untuk menjawab pertanyaan dalam bentuk kuesioner, dan setiap pertanyaan dinilai dari lima tingkat, seperti yang disajikan pada Tabel 2.9.

Tabel 2. 9 Tabel Skala Penilaian

Skala Penilaian	Keterangan
1	Sangat Tidak Setuju
2	Tidak Setuju
3	Cukup
4	Setuju
5	Sangat Setuju

Dari hasil tabulasi skor seluruh responden pada 10 aspek tersebut, diperoleh Total skor aktual (dari semua responden dan aspek) = 1.622 poin.

1) Perhitungan Skor Maksimal

Skor maksimal dihitung berdasarkan asumsi bahwa seluruh responden memberikan nilai tertinggi yaitu 5 pada setiap aspek. Maka:

Skor Maks =

Jumlah Responden $\times$ Jumlah Aspek $\times$ Skor Maks per Aspek

$$\text{Skor Maks} = 37 \times 10 \times 5 = \mathbf{1.850}$$

2) Perhitungan Persentase *Usability*

Persentase *usability* menggambarkan sejauh mana aplikasi memenuhi harapan dan kenyamanan pengguna berdasarkan penilaian mereka.

Persentase ini dihitung dengan rumus:

$$\text{Persentase Usability} = \frac{\text{Total Skor Aktual}}{\text{Skor Maksimal}} \times 100\%$$

$$\text{Persentase Usability} = \frac{1.622}{1850} \times 100\% = \mathbf{87,68\%}$$

Berdasarkan hasil pengolahan data tersebut, diperoleh nilai usabilitas sebesar 87,68%, yang menunjukkan bahwa aplikasi RumahTBC memiliki tingkat kegunaan (*usability*) yang sangat baik. Hal ini mencerminkan bahwa secara umum, pengguna merasa nyaman, terbantu, dan mudah dalam menggunakan aplikasi untuk keperluan pemantauan tuberkulosis.

2.2 Kesimpulan dan Saran

2.2.1 Kesimpulan

Berdasarkan hasil penelitian dapat ditarik beberapa kesimpulan sebagai berikut:

1. Aplikasi "RumahTBC" telah berhasil dikembangkan sebagai solusi digital berbasis Android yang berfungsi untuk membantu penanganan tuberkulosis secara komprehensif. Aplikasi ini mengintegrasikan fitur deteksi dini, pemantauan kepatuhan, dan edukasi dalam satu platform.
2. Fitur deteksi dini tuberkulosis berhasil dibangun menggunakan metode sistem pakar berbasis gejala klinis yang telah divalidasi oleh dokter spesialis paru.

Fitur ini dapat berfungsi sebagai alat skrining awal yang informatif dan edukatif bagi masyarakat untuk mengidentifikasi risiko TBC secara mandiri.

3. Sistem pemantauan pengobatan yang terintegrasi telah berhasil diimplementasikan untuk mendukung kepatuhan pasien. Fitur ini mencakup pengingat jadwal minum obat dan kontrol ke rumah sakit, serta melibatkan peran keluarga sebagai Pengawas Minum Obat (PMO) untuk memastikan terapi berjalan teratur.
4. Aplikasi “RumahTBC” berhasil menyediakan akses terhadap informasi kesehatan yang akurat melalui fitur artikel, yang berperan dalam meningkatkan literasi dan kesadaran masyarakat terkait pencegahan serta penanganan tuberkulosis secara efektif. Selain itu, aplikasi ini juga mampu mengintegrasikan data riwayat pemeriksaan dan pengobatan, sehingga memungkinkan pasien untuk secara mandiri melacak dan memantau perkembangan pengobatan mereka secara lebih sistematis dan terarah.
5. Pengujian fungsionalitas pada fitur-fitur dalam aplikasi RUMAHTBC menunjukkan hasil yang optimal. Sementara itu, pengujian *usability* mencerminkan tingkat kepuasan pengguna yang tinggi terhadap antarmuka serta kemudahan penggunaan fitur-fitur aplikasi. Mayoritas responden menilai aplikasi ini bermanfaat dalam proses skrining, pemantauan pengobatan, dan kepatuhan pengobatan tuberkulosis, dengan tingkat kepuasan mencapai 87,68%.

2.2.2 Saran

Untuk pengembangan aplikasi "RumahTBC" di masa mendatang, terdapat beberapa saran yang dapat dipertimbangkan:

1. Integrasi dengan Sistem Informasi Rumah Sakit (SIMRS): Hal ini akan memungkinkan sinkronisasi data pasien secara *real-time* dan mempermudah tenaga kesehatan dalam melakukan pemantauan.
2. Pengembangan *Multi-Platform*: Mengingat aplikasi saat ini hanya tersedia untuk platform Android, disarankan untuk mengembangkan versi aplikasi untuk platform lain seperti iOS atau berbasis web.
3. Pengembangan Fitur Telekonsultasi: Menambahkan fitur konsultasi daring dengan tenaga kesehatan atau dokter spesialis. Fitur ini dapat menjadi jembatan bagi pasien yang memerlukan nasihat medis cepat tanpa harus datang langsung ke fasilitas kesehatan.